

BỘ GIÁO DỤC VÀ ĐÀO TẠO
TRƯỜNG ĐẠI HỌC SƯ PHẠM KỸ THUẬT TP.HỒ CHÍ MINH
KHOA ĐIỆN – ĐIỆN TỬ
BỘ MÔN KỸ THUẬT MÁY TÍNH – VIỄN THÔNG



HCMUTE

TIỂU LUẬN MÔN HỌC
KIẾN TRÚC VÀ GIAO THỨC IOT

THIẾT KẾ HỆ THỐNG GIÁM SÁT NGỌN LỬA

Giáo viên hướng dẫn: Ths. Trịnh Quốc Thanh

Nhóm thực hiện: Nhóm 7

Mã lớp học: ITAP338264

TP.HỒ CHÍ MINH – THÁNG 10 NĂM 2025

BỘ GIÁO DỤC VÀ ĐÀO TẠO
TRƯỜNG ĐẠI HỌC SƯ PHẠM KỸ THUẬT TP.HỒ CHÍ MINH
KHOA ĐIỆN – ĐIỆN TỬ
BỘ MÔN KỸ THUẬT MÁY TÍNH – VIỄN THÔNG
◆◆◆◆◆



HCMUTE

TIỂU LUẬN MÔN HỌC
KIẾN TRÚC VÀ GIAO THỨC IOT
THIẾT KẾ HỆ THỐNG GIÁM SÁT NGỌN LỬA

GVHD: Ths. Trịnh Quốc Thanh

SVTH: Nhóm 7

1: Lê Nhật Nam

MSSV: 23139029

TP.HỒ CHÍ MINH – THÁNG 10 NĂM 2025

NHẬN XÉT CỦA GIẢNG VIÊN

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

Điểm:

KÝ TÊN

DANH MỤC HÌNH VẼ

Hình 1: Mô hình tổng quát của Internet of Things, thể hiện các tầng: Cảm nhận – Mạng – Xử lý – Ứng dụng	6
Hình 2 : Vi điều khiển STM32F103C8T6	13
Hình 3: Các thông số chân ESP32	16
Hình 4: Buzzer	18
Hình 5: Bóng LED	19
Hình 6: Cảm biến IR (Flame Sensor).....	19
Hình 7 : Sơ đồ nguyên lý của cảm biến IR.....	20
Hình 8 : Sơ đồ kết nối của Cảm biến nhiệt độ DS18B20 dây mềm	22
Hình 9: Giao tiếp theo chuẩn UART	22
Hình 10: Hệ thống truyền dữ liệu bất đồng bộ	25
Hình 11: Giao tiếp với giao thức MQTT	28
Hình 12: Cấu trúc kết hợp HTML – CSS – JavaScript trong phát triển giao diện web	30
Hình 13: Logo HTML	30
Hình 14: Logo CSS	32
Hình 15: Logo Javasript	33
Hình 16: Sơ đồ khối tổng quát hệ thống phát hiện cháy.....	36
Hình 17: Sơ đồ kết nối phần cảm biến với STM32	37
Hình 18: Sơ đồ truyền thông giữa STM32, ESP32 và MQTT Broker	39
Hình 19: Giao diện hiển thị dữ liệu và cảnh báo cháy thời gian thực	40
Hình 20: Lưu đồ chương trình chính của hệ thống.....	41
Hình 22: Ảnh chụp Dashboard ở hai trạng thái Safe và Fire Detected – chèn ảnh	48

DANH MỤC BẢNG HIỆU

Bảng 1: Một số loại cảm biến nhiệt phổ biến hiện tại	10
Bảng 2: Thông số kỹ thuật của STM32F103C8T6.....	14
Bảng 3: Thông số kỹ thuật của ESP32.....	16
Bảng 4: Thông số kỹ thuật của Buzzer	17
Bảng 5: Thông số kỹ thuật của IR	20
Bảng 6 : Thông số của Cảm biến nhiệt độ DS18B20 dây mềm.....	21

MỤC LỤC

NHẬN XÉT CỦA GIẢNG VIÊN	3
-------------------------------	---

DANH MỤC HÌNH VẼ	i
DANH MỤC BẢNG HIỆU	ii
MỤC LỤC	ii
CHƯƠNG I: GIỚI THIỆU TỔNG QUAN VỀ ĐỀ TÀI	1
1.1. Giới thiệu	1
1.2. Mục tiêu đề tài	1
1.3. Đối tượng, phạm vi đề tài	2
1.4. Phương pháp nghiên cứu	3
1.5. Kết cấu báo cáo	4
CHƯƠNG 2: CƠ SỞ LÝ THUYẾT	5
2.1. Khái niệm cơ bản.....	5
2.1.1. Khái niệm về hệ thống nhúng	5
2.1.2. Khái niệm về IoT	5
2.1.3. Ứng dụng IoT trong hệ thống giám sát và cảnh báo cháy.....	6
2.2. Cảm biến phát hiện cháy.....	6
2.2.1. Khái niệm về cảm biến ngọn lửa IR	7
2.2.2. Nguyên lý hoạt động của cảm biến IR	7
2.2.3. Ứng dụng và giới hạn của cảm biến ngọn lửa	8
2.3. Cảm biến nhiệt độ.....	9
2.3.1. Khái niệm về đo nhiệt độ và cảm biến nhiệt.....	9
2.3.2. Thang đo nhiệt độ	9
2.3.3. Một số loại cảm biến nhiệt phổ biến	10
2.3.4. Cảm biến nhiệt độ dùng trong đề tài	11
2.4. Giới thiệu phần cứng sử dụng.....	12
2.4.1. Vi điều khiển STM32F103C8T6.....	12
2.4.2. Vi điều khiển ESP32.....	15

2.4.3. Còi báo (Buzzer).....	17
2.4.4. Đèn LED cảnh báo.....	18
2.4.5. Cảm biến IR (Flame Sensor).....	19
2.4.6. Cảm biến nhiệt độ DS18B20S	20
2.5. Các chuẩn truyền dữ liệu	22
2.5.1. Chuẩn giao tiếp UART (giữa STM32 và ESP32).....	22
2.5.2. Chuẩn giao tiếp 1-Wire.....	24
2.5.4. Chuẩn truyền Wifi	27
2.6. Giao thức MQTT	27
2.7. UI (User interface) và UX (User experience).....	30
2.7.1. HTML	30
2.7.2. CSS	31
2.7.3. Javascript (JS).....	33
CHƯƠNG 3: THIẾT KẾ VÀ THI CÔNG MÔ HÌNH	35
3.1. Phân tích yêu cầu hệ thống	35
3.2. Sơ đồ khối và nguyên lý hoạt động.....	35
3.2.1. Sơ đồ khối tổng quát.....	35
3.2.2. Tầng cảm nhận	37
3.2.3. Tầng xử lý trung gian (STM32)	37
3.2.4. Tầng xử lý chính (ESP32).....	38
3.2.5. Tầng truyền thông.....	38
3.2.6. Tầng ứng dụng.....	39
3.2.7. Nguyên lý hoạt động tổng quát	40
3.3. Lập trình hệ thống	40
3.3.1. Yêu cầu của phần mềm.....	40
3.3.2. Lưu đồ giải thuật chương trình chính của hệ thống	41

3.3.3. Khung dữ liệu UART STM32 → ESP32	42
3.3.4. Chủ đề (topic) MQTT và payload	42
3.4. Thiết kế giao diện giám sát	43
3.4.1. Cấu trúc giao diện của người dung.....	43
3.4.2. Cấu trúc HTML tổng quát.....	44
3.4.3. Chức năng điều khiển	45
CHƯƠNG 4: THỰC NGHIỆM, KIỂM CHỨNG VÀ ĐÁNH GIÁ HỆ THỐNG	46
4.1. Mô hình hệ thống thực tế	46
4.2. Quy trình thực nghiệm.....	46
4.3. Kết quả truyền dữ liệu và hiển thị.....	47
4.4. Đánh giá kết quả.....	48
CHƯƠNG 5: KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN.....	49
5.1. Kết luận.....	49
5.2. Đánh giá chung.....	49
5.3. Hướng phát triển.....	50
5.4. Kết luận chung.....	50
TÀI LIỆU THAM KHẢO	51
PHỤ LỤC	53
CODE STM32.....	53
CODE WEB.....	60

CHƯƠNG I: GIỚI THIỆU TỔNG QUAN VỀ ĐỀ TÀI

1.1. Giới thiệu

Trong bối cảnh công nghiệp hóa và đô thị hóa diễn ra nhanh chóng, nguy cơ cháy nổ ngày càng trở thành vấn đề đáng lo ngại, đặc biệt tại các khu dân cư, nhà kho, phòng thí nghiệm và cơ sở sản xuất nhỏ. Các vụ hỏa hoạn không chỉ gây thiệt hại lớn về người và tài sản mà còn ảnh hưởng nghiêm trọng đến môi trường và đời sống xã hội. Một trong những nguyên nhân chủ yếu là việc phát hiện cháy chậm hoặc hệ thống cảnh báo hoạt động chưa hiệu quả, dẫn đến phản ứng xử lý không kịp thời.

Hiện nay, nhiều hệ thống cảnh báo cháy vẫn chủ yếu sử dụng cảm biến đơn lẻ như cảm biến khói hoặc cảm biến nhiệt. Tuy nhiên, các loại cảm biến này dễ bị ảnh hưởng bởi các yếu tố môi trường như ánh sáng mạnh, thay đổi nhiệt độ đột ngột hay hơi nước, gây ra hiện tượng báo động giả và làm giảm độ tin cậy của hệ thống.

Trước thực tế đó, nhóm thực hiện đề tài “Hệ thống phát hiện ngọn lửa bằng IR kết hợp xác nhận bằng nhiệt độ qua STM32 và MQTT” nhằm xây dựng một giải pháp phát hiện cháy sớm, chính xác và có khả năng giám sát từ xa. Trong hệ thống này, STM32 đảm nhiệm vai trò thu thập dữ liệu từ cảm biến hồng ngoại (IR Flame Sensor) và cảm biến nhiệt độ, sau đó gửi dữ liệu về ESP32 – vi điều khiển trung tâm để xử lý, phân tích và đưa ra cảnh báo.

ESP32 đồng thời chịu trách nhiệm truyền dữ liệu cảnh báo theo thời gian thực qua giao thức MQTT lên máy chủ hoặc thiết bị di động, đáp ứng xu hướng Internet of Things (IoT) hiện nay. Nhờ đó, người dùng có thể giám sát tình trạng an toàn của khu vực mọi lúc, mọi nơi, góp phần nâng cao hiệu quả phòng cháy chữa cháy và bảo vệ tài sản.

1.2. Mục tiêu đề tài

Đề tài hướng đến việc thiết kế và triển khai một hệ thống phát hiện và cảnh báo cháy thông minh, có khả năng xử lý nhanh, chính xác và giám sát từ xa qua Internet. Hệ thống được xây dựng trên nền ESP32 làm trung tâm xử lý, đồng thời sử dụng STM32 như một mô-đun thu thập dữ liệu từ các cảm biến (IR Flame

Sensor và cảm biến nhiệt độ) để gửi về ESP32 thông qua giao tiếp nối tiếp (UART).

ESP32 đảm nhiệm vai trò xử lý dữ liệu, xác định tình trạng cháy và truyền thông tin cảnh báo lên máy chủ hoặc điện thoại di động thông qua giao thức MQTT, phù hợp với xu hướng Internet of Things (IoT) hiện nay.

Hệ thống được kỳ vọng giúp phát hiện cháy sớm, giảm thiểu báo động giả, và nâng cao khả năng giám sát, quản lý an toàn từ xa, cụ thể:

Thiết kế hệ thống cảm biến gồm cảm biến hồng ngoại (IR) và cảm biến nhiệt độ, kết nối với STM32 để thu thập tín hiệu môi trường.

Lập trình STM32 để xử lý sơ bộ và truyền dữ liệu cảm biến về ESP32 qua UART.

Xây dựng chương trình xử lý trung tâm trên ESP32 nhằm xác nhận điều kiện cháy, điều khiển thiết bị cảnh báo (LED, còi buzzer) và gửi cảnh báo qua MQTT.

Phát triển nền tảng giám sát từ xa, cho phép người dùng theo dõi tình trạng khu vực theo thời gian thực qua ứng dụng hoặc web.

Kiểm thử và đánh giá hệ thống trong nhiều điều kiện thực tế để đo độ nhạy, độ chính xác và độ ổn định khi phát hiện cháy.

1.3. Đối tượng, phạm vi đề tài

Đối tượng nghiên cứu chính của đề tài là một hệ thống nhúng phát hiện và cảnh báo cháy thông minh, sử dụng cảm biến ngọn lửa IR, cảm biến nhiệt độ, vi điều khiển STM32 và ESP32 để thực hiện thu thập, xử lý và truyền dữ liệu cảnh báo qua giao thức MQTT.

Trong hệ thống này, STM32 đóng vai trò là bộ thu thập dữ liệu cảm biến, đọc tín hiệu từ cảm biến ngọn lửa (IR) và cảm biến nhiệt độ thông qua ADC hoặc giao tiếp I²C/1-Wire, sau đó truyền dữ liệu sang ESP32 qua giao tiếp UART. ESP32 là bộ xử lý trung tâm, đảm nhận việc phân tích dữ liệu, xác định điều kiện cháy, điều khiển thiết bị cảnh báo (đèn LED, còi buzzer), đồng thời gửi thông tin cảnh báo lên MQTT Broker (đặt trên server nội bộ hoặc cloud) để phục vụ giám sát từ xa thời gian thực.

Phạm vi nghiên cứu tập trung vào môi trường vừa và nhỏ như phòng học, phòng thí nghiệm, nhà bếp hoặc kho chứa, tiến hành thử nghiệm trong các điều kiện ánh sáng và nhiệt độ khác nhau để đánh giá độ chính xác và độ tin cậy của hệ thống. Hệ thống hiện chưa tích hợp chức năng tự động chữa cháy hoặc xử lý hình ảnh, nhưng được thiết kế mở để có thể mở rộng trong các giai đoạn nghiên cứu sau.

1.4. Phương pháp nghiên cứu

a) Nghiên cứu tài liệu

Thu thập và tổng hợp các tài liệu liên quan đến cảm biến ngọn lửa IR, cảm biến nhiệt độ, vi điều khiển STM32, ESP32 và giao thức truyền thông MQTT.

Tham khảo datasheet của các linh kiện, tài liệu hướng dẫn kỹ thuật của STM32Cube IDE và Arduino IDE (cho ESP32), cùng các bài báo và nghiên cứu khoa học về hệ thống phát hiện cháy dựa trên IoT.

b) Phân tích và thiết kế hệ thống

Phân tích yêu cầu hệ thống: phát hiện cháy nhanh, giảm cảnh báo sai, truyền dữ liệu ổn định và phản hồi kịp thời.

Xác định cấu trúc hệ thống gồm:

Phần cứng: cảm biến IR, cảm biến nhiệt độ, STM32, ESP32, còi buzzer và đèn LED.

Phần mềm: chương trình nhúng cho STM32 và ESP32, cùng ứng dụng MQTT Broker – Client.

Thiết kế sơ đồ khối hệ thống, luồng dữ liệu và thuật toán xử lý hai giai đoạn (IR + Nhiệt độ) để xác định điều kiện cháy chính xác.

c) Thực nghiệm và mô phỏng

Lắp ráp mô hình, cấu hình giao tiếp ADC, UART, I²C trên STM32 và ESP32.

Hiệu chỉnh ngưỡng phát hiện dựa trên thử nghiệm thực tế với nguồn lửa an toàn (nến, bếp cồn).

Thử nghiệm khả năng truyền dữ liệu MQTT và cảnh báo thời gian thực thông qua mạng Wi-Fi, đánh giá độ trễ và độ ổn định của hệ thống.

1.5. Kết cấu báo cáo

Báo cáo được trình bày theo bố cục như sau:

Chương 1: Tổng quan – Trong chương này, chúng ta sẽ trình bày về tính cấp thiết của đề tài, cũng như phân tích xu hướng và tình hình khoa học và công nghệ hiện nay. Sự phát triển của ngành công nghiệp và nhu cầu phục vụ cuộc sống hàng ngày được xem là những lý do quan trọng khi chọn đề tài và xác định mục tiêu nghiên cứu cho đề tài này.

Chương 2: Cơ sở lý thuyết – Trình bày tổng quan về thành phần, đặc tính cũng như chức năng của từng loại linh kiện ở phần cứng trong hệ thống, từ đó sẽ kết hợp lại để có thể xây dựng nên một hệ thống hoàn chỉnh về mô hình.

Chương 3: Nghiên cứu và thiết kế hệ thống – Từ yêu cầu của đề tài, ta thiết kế sơ đồ cho hệ thống(Sơ đồ khối, Sơ đồ nguyên lý...). Đưa ra các phương pháp xử lý và chỉnh sửa từ đó thiết kế nên mô hình.

Chương 4: Thực nghiệm và đánh giá – mô tả quá trình triển khai, hiệu chuẩn, đo đạc và phân tích kết quả.

Chương 5: Kết luận và hướng phát triển – tổng hợp kết quả đạt được và đề xuất hướng cải tiến trong tương lai.

CHƯƠNG 2: CƠ SỞ LÝ THUYẾT

2.1. Khái niệm cơ bản

2.1.1. Khái niệm về hệ thống nhúng

Hệ thống nhúng (Embedded System) là một hệ thống máy tính chuyên dụng được tích hợp vào trong các thiết bị lớn hơn nhằm thực hiện một chức năng cụ thể. Khác với máy tính đa năng, hệ thống nhúng thường có ràng buộc về thời gian thực, được tối ưu hóa về chi phí, kích thước và độ tin cậy. Chúng xuất hiện rộng rãi trong đời sống hiện đại, từ các thiết bị điện tử tiêu dùng như điện thoại, máy giặt, đến các hệ thống công nghiệp, y tế, ô tô và hàng không. Về mặt kỹ thuật, hệ thống nhúng thường sử dụng vi điều khiển hoặc vi xử lý tích hợp bộ nhớ và giao tiếp ngoại vi, phần mềm điều khiển thường là firmware lưu trong bộ nhớ ROM hoặc flash. Kiến trúc phần mềm của chúng có thể đơn giản như vòng lặp điều khiển hoặc phức tạp hơn với hệ điều hành thời gian thực (RTOS), hỗ trợ đa nhiệm và quản lý tài nguyên. Để đảm bảo độ tin cậy, hệ thống nhúng thường được thiết kế với khả năng tự phục hồi, sử dụng các kỹ thuật như watchdog timer và lập trình miễn nhiễm lỗi. Với vai trò ngày càng quan trọng trong tự động hóa và kết nối vạn vật (IoT), hệ thống nhúng là nền tảng không thể thiếu trong sự phát triển của công nghệ hiện đại.

2.1.2. Khái niệm về IoT

Internet of Things (IoT), hay còn gọi là “Internet vạn vật”, là mô hình trong đó các thiết bị vật lý được tích hợp cảm biến, bộ xử lý và khả năng kết nối mạng, cho phép chúng tự động thu thập, trao đổi và xử lý dữ liệu mà không cần sự can thiệp của con người. Mục tiêu của IoT là hình thành một mạng lưới thông minh, nơi các thiết bị có thể giao tiếp với nhau và với hệ thống trung tâm để tối ưu hóa khả năng giám sát và điều khiển.

IoT được ứng dụng trong nhiều lĩnh vực như nông nghiệp thông minh, nhà ở thông minh, y tế, công nghiệp tự động hóa và đặc biệt là trong các hệ thống giám sát an toàn. Trong bối cảnh công nghiệp hóa hiện nay, việc ứng dụng IoT giúp giảm chi phí, nâng cao hiệu quả vận hành và cho phép người dùng giám sát các thông số môi trường theo thời gian thực.

MÔ HÌNH 4 LỚP IOT

VTI SOLUTIONS



Hình 1: Mô hình tổng quát của Internet of Things, thể hiện các tầng: Cảm nhận – Mạng – Xử lý – Ứng dụng

(Nguồn: VTI Solutions)

2.1.3. Ứng dụng IoT trong hệ thống giám sát và cảnh báo cháy

Ứng dụng công nghệ IoT (Internet of Things) trong hệ thống giám sát và cảnh báo cháy đang trở thành xu hướng tất yếu trong bối cảnh đô thị hóa và nhu cầu an toàn ngày càng tăng. Hệ thống này sử dụng các cảm biến như nhiệt độ, khói, khí gas và camera để thu thập dữ liệu môi trường theo thời gian thực, từ đó phát hiện sớm nguy cơ cháy nổ. Khi có dấu hiệu bất thường, hệ thống sẽ gửi cảnh báo tức thời đến người dùng qua điện thoại, máy tính hoặc còi báo động, đồng thời có thể tự động kích hoạt các biện pháp ứng phó như hệ thống chữa cháy hoặc thông báo cho lực lượng cứu hộ. Ngoài ra, IoT còn giúp giám sát trạng thái thiết bị phòng cháy chữa cháy (PCCC), hỗ trợ bảo trì định kỳ và nâng cao hiệu quả quản lý. Nhờ khả năng kết nối liên tục 24/7 và phân tích dữ liệu chính xác, hệ thống báo cháy thông minh ứng dụng IoT góp phần giảm thiểu thiệt hại về người và tài sản, đồng thời nâng cao năng lực phản ứng trong các tình huống khẩn cấp.

2.2. Cảm biến phát hiện cháy

Cảm biến phát hiện cháy là một trong những thành phần quan trọng trong hệ thống cảnh báo cháy tự động. Mục đích chính của cảm biến này là phát hiện

sớm dấu hiệu của ngọn lửa thông qua bức xạ ánh sáng phát ra trong quá trình cháy.

Trong đề tài này, nhóm sử dụng cảm biến ngọn lửa hồng ngoại (IR Flame Sensor) để phát hiện tia hồng ngoại đặc trưng của ngọn lửa, sau đó kết hợp với cảm biến nhiệt độ để xác nhận điều kiện cháy thực sự, giúp giảm báo động giả.

2.2.1. Khái niệm về cảm biến ngọn lửa IR

Cảm biến ngọn lửa hồng ngoại (Infrared Flame Sensor) là thiết bị điện tử có khả năng phát hiện tia hồng ngoại (IR) trong dải bước sóng đặc trưng mà ngọn lửa phát ra, thường nằm trong khoảng 760nm – 1100nm.

Khi có ngọn lửa, năng lượng bức xạ ở vùng hồng ngoại được cảm biến thu nhận và chuyển đổi thành tín hiệu điện áp hoặc tín hiệu số, gửi đến vi điều khiển để xử lý.

Module cảm biến IR thường bao gồm:

Cảm biến quang IR (photodiode hoặc phototransistor) – phần tử phát hiện ánh sáng.

Mạch khuếch đại và so sánh – để khuếch đại tín hiệu thu được và tạo ngõ ra mức logic (HIGH/LOW).

Biến trở điều chỉnh độ nhạy (Sensitivity potentiometer) – cho phép hiệu chỉnh khoảng phát hiện phù hợp.

2.2.2. Nguyên lý hoạt động của cảm biến IR

Cảm biến ngọn lửa hồng ngoại (IR Flame Sensor) hoạt động dựa trên nguyên lý phát hiện bức xạ hồng ngoại (Infrared Radiation) phát ra từ ngọn lửa. Khi các vật chất cháy, chúng sinh ra bức xạ điện từ trong dải bước sóng đặc trưng khoảng 760 nm đến 1100 nm, nằm trong vùng hồng ngoại gần (Near-Infrared). Cảm biến IR được thiết kế để nhận biết loại bức xạ này và chuyển đổi nó thành tín hiệu điện để vi điều khiển xử lý.

Cấu tạo của cảm biến gồm điốt quang hồng ngoại (IR photodiode) hoặc transistor quang (phototransistor), kết hợp với mạch khuếch đại và bộ lọc tín hiệu. Khi ngọn lửa xuất hiện trong phạm vi cảm ứng, năng lượng hồng ngoại chiếu tới

bề mặt cảm biến làm thay đổi điện áp đầu ra. Mức điện áp này tỉ lệ thuận với cường độ bức xạ thu được.

Tín hiệu tương tự (analog) hoặc số (digital) từ cảm biến sẽ được gửi đến vi điều khiển (trong hệ thống là STM32). Tại đây, chương trình sẽ đọc giá trị đầu vào và so sánh với một ngưỡng xác định trước để quyết định xem có ngọn lửa xuất hiện hay không.

Để đảm bảo độ chính xác, cảm biến IR thường được tích hợp bộ lọc quang học giúp loại bỏ nhiễu từ ánh sáng môi trường (như ánh sáng mặt trời hoặc đèn LED), chỉ cho phép dải bước sóng của ngọn lửa đi qua. Ngoài ra, một số module còn có biến trở điều chỉnh độ nhạy, cho phép người dùng hiệu chỉnh phạm vi phát hiện phù hợp với từng ứng dụng.

2.2.3. Ứng dụng và giới hạn của cảm biến ngọn lửa

Ứng dụng

- Hệ thống phát hiện cháy tự động trong nhà kho, phòng thí nghiệm, nhà bếp.
- Robot dò lửa – chữa cháy tự động (Fire Fighting Robot).
- Ứng dụng giám sát an toàn trong công nghiệp.
- Phát hiện nguồn nhiệt hoặc lửa trong dây chuyền sản xuất.

Giới hạn

- Dễ bị nhiễu bởi ánh sáng mạnh hoặc ánh sáng mặt trời trực tiếp, do cùng dải bước sóng.
- Khoảng cách phát hiện ngắn, thường 0.8 – 1.5 mét.
- Không phát hiện được khói hoặc cháy âm ỉ, vì cần ánh sáng trực tiếp từ ngọn lửa.
- Không phân biệt được nguồn phát sáng nhân tạo (ví dụ: đèn hàn, đèn sưởi).

=> Do đó, trong đề tài này cảm biến ngọn lửa IR được kết hợp với cảm biến nhiệt độ, giúp hệ thống xác nhận chính xác hơn tình trạng cháy thật, hạn chế báo động giả.

2.3. Cảm biến nhiệt độ

2.3.1. Khái niệm về đo nhiệt độ và cảm biến nhiệt

Đo nhiệt độ

Đo nhiệt độ là quá trình xác định độ "nóng" hoặc "lạnh" của một vật thể, môi trường, hoặc hệ thống. Nhiệt độ là đại lượng vật lý dùng để định lượng năng lượng chuyển động hỗn loạn của các phân tử và nguyên tử trong vật chất. Việc đo lường này đóng vai trò then chốt trong nhiều lĩnh vực khoa học, công nghiệp (kiểm soát quy trình), y tế và đời sống (giám sát môi trường, khí hậu).

Cảm biến nhiệt độ

Cảm biến nhiệt độ là thiết bị chuyển đổi sự thay đổi của nhiệt độ vật lý thành một tín hiệu điện có thể đo lường được (thường là điện áp, dòng điện, hoặc sự thay đổi điện trở).

Nguyên lý hoạt động chung: Cảm biến nhiệt sử dụng sự thay đổi của một đặc tính vật lý nào đó (như điện trở, điện áp tiếp xúc, thể tích, v.) của vật liệu khi nhiệt độ thay đổi để tạo ra tín hiệu đầu ra.

2.3.2. Thang đo nhiệt độ

Việc đo lường nhiệt độ được chuẩn hóa thông qua các thang đo quốc tế, trong đó ba thang đo phổ biến nhất là: Celsius ($^{\circ}C$), được sử dụng rộng rãi trong hầu hết các ứng dụng hàng ngày và kỹ thuật, với $0^{\circ}C$ là điểm đóng băng và $100^{\circ}C$ là điểm sôi của nước tinh khiết (áp suất tiêu chuẩn). Fahrenheit ($^{\circ}F$), chủ yếu được sử dụng tại Hoa Kỳ. Cuối cùng là Kelvin (K), là đơn vị cơ bản của nhiệt độ trong Hệ đo lường quốc tế (SI), được dùng trong các tính toán khoa học, với $0K$ là Độ không tuyệt đối (Absolute Zero), tương đương với $-273.15^{\circ}C$

2.3.3. Một số loại cảm biến nhiệt phổ biến

Bảng 1: Một số loại cảm biến nhiệt phổ biến hiện tại

Loại Cảm Biến	Ký hiệu	Nguyên lý hoạt động	Phạm vi đo điện hình	Độ chính xác/Ứng dụng
Nhiệt điện trở (Thermistor)	NTC / PTC	Điện trở thay đổi mạnh (không tuyến tính) theo nhiệt độ (NTC: giảm; PTC: tăng).	$-50^{\circ}C$ đến $150^{\circ}C$	Độ nhạy cao, giá thành thấp, giám sát nhiệt độ trong phạm vi hẹp.
Điện trở nhiệt (RTD)	Pt100	Điện trở của kim loại (thường là Platin) tăng gần như tuyến tính với nhiệt độ.	$-200^{\circ}C$ đến $600^{\circ}C$	Độ chính xác cao, độ ổn định lâu dài, ứng dụng trong công nghiệp.

Cặp nhiệt điện (Thermocouple)	Type K, J, T, ...	Dựa trên hiệu ứng Seebeck, tạo ra điện áp nhỏ tỷ lệ với chênh lệch nhiệt độ.	$-200^{\circ}C$ đến $2300^{\circ}C$	Phạm vi đo rộng nhất, độ bền cơ học cao, tốc độ đáp ứng nhanh.
Cảm biến nhiệt IC (IC Sensor)	LM35, DS18B20	Sử dụng các đặc tính của linh kiện bán dẫn (diode/transistor) để tạo ra tín hiệu điện áp/kỹ thuật số tuyến tính.	$-55^{\circ}C$ đến $150^{\circ}C$	Tín hiệu tuyến tính, dễ tích hợp với vi điều khiển, ứng dụng trong hệ thống nhúng.

2.3.4. Cảm biến nhiệt độ dùng trong đề tài

Nguyên lý hoạt động và ứng dụng

Mặc dù được gọi là cảm biến nhiệt, cảm biến lửa hồng ngoại thực chất là một thiết bị phát hiện bức xạ điện từ phát ra từ ngọn lửa. Nguyên lý cốt lõi là việc sử dụng một hoặc nhiều điốt quang nhạy cảm với dải tần số hồng ngoại (IR) cụ thể, thường là bước sóng từ 760nm đến 1100nm (dải cận hồng ngoại). Ngọn lửa phát ra một lượng lớn bức xạ trong dải này. Khi cường độ bức xạ vượt qua ngưỡng đã đặt, cảm biến sẽ kích hoạt tín hiệu đầu ra.

Lý do lựa chọn

Phát hiện tức thì: Cảm biến phản ứng rất nhanh với sự xuất hiện của ngọn lửa, cho phép hệ thống cảnh báo và ứng phó tức thời với nguy cơ hỏa hoạn.

Hoạt động không tiếp xúc: Thiết bị không cần tiếp xúc vật lý với ngọn lửa, đảm bảo an toàn và độ bền.

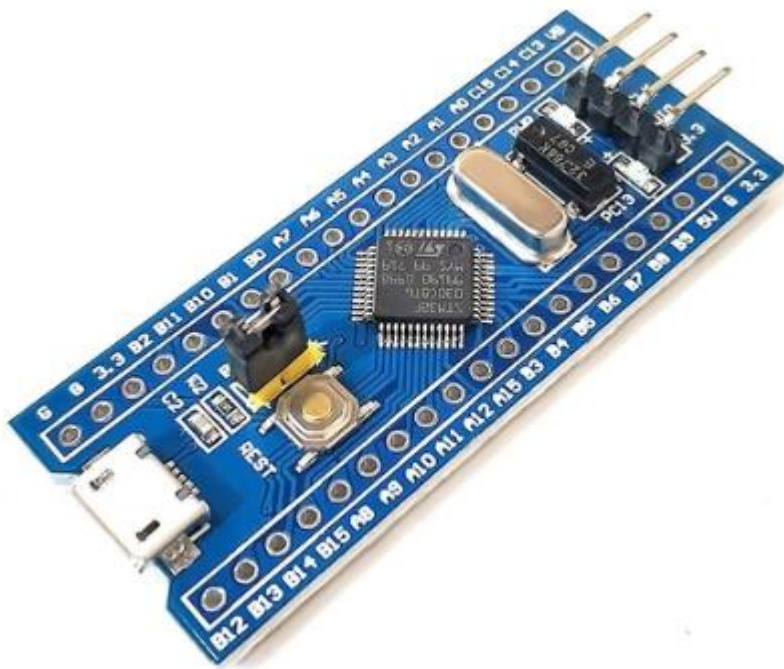
Đầu ra: Cảm biến thường cung cấp cả đầu ra analog (tỷ lệ với cường độ bức xạ) và đầu ra kỹ thuật số (tín hiệu ngưỡng), dễ dàng tích hợp với vi điều khiển để vừa đo cường độ tương đối, vừa cảnh báo nhanh.

2.4. Giới thiệu phần cứng sử dụng

2.4.1. Vi điều khiển STM32F103C8T6

Vi điều khiển STM32 là dòng vi điều khiển 32-bit do STMicroelectronics phát triển, dựa trên kiến trúc ARM Cortex-M3. Dòng sản phẩm này nổi bật nhờ hiệu năng xử lý cao, tiêu thụ năng lượng thấp, khả năng tích hợp nhiều ngoại vi và độ tin cậy trong vận hành. Trong số đó, STM32F103C8T6, thường được biết đến với tên gọi “Blue Pill”, là một trong những phiên bản phổ biến nhất nhờ giá thành hợp lý, cấu hình mạnh và dễ dàng lập trình trên nền tảng STM32Cube IDE.

Trong đề tài này, vi điều khiển STM32F103C8T6 được lựa chọn làm bộ xử lý trung gian, đảm nhận nhiệm vụ thu thập dữ liệu từ cảm biến hồng ngoại và cảm biến nhiệt độ, xử lý sơ bộ tín hiệu, sau đó gửi kết quả phát hiện cháy sang ESP32 để truyền dữ liệu lên hệ thống giám sát MQTT.



Hình 2 : Vi điều khiển STM32F103C8T6

(Nguồn: Imaker.vn)

Đặc điểm

Hiệu năng xử lý cao: sử dụng lõi ARM Cortex-M3 hoạt động ở tần số tối đa 72MHz, giúp xử lý nhanh các phép tính và tín hiệu thời gian thực.

Bộ nhớ đủ lớn: trang bị 64KB Flash và 20 KB SRAM, đáp ứng tốt cho các ứng dụng cần lưu trữ và xử lý dữ liệu.

Hỗ trợ nhiều giao thức truyền thông: tích hợp sẵn UART, SPI, I²C, CAN, USB, cho phép dễ dàng giao tiếp với cảm biến và module ngoại vi.

Tích hợp bộ chuyển đổi tương tự – số (ADC): 12-bit với 16 kênh độc lập, phục vụ cho việc đo tín hiệu từ cảm biến.

Nhiều chân GPIO linh hoạt: có 37 chân vào/ra, có thể cấu hình cho nhiều chức năng khác nhau như đọc cảm biến, điều khiển đèn, còi, relay,...

Tích hợp Timer và PWM: hỗ trợ điều khiển thiết bị ngoại vi với độ chính xác cao.

Hỗ trợ nạp và gỡ lỗi dễ dàng: tương thích với bộ nạp ST-Link V2 qua chuẩn SWD (Serial Wire Debug).

Hoạt động ổn định trong dải điện áp rộng: từ 2.0V đến 3.6V, đảm bảo độ bền và tính linh hoạt cao.

Cộng đồng phát triển lớn: có nhiều thư viện, ví dụ mẫu và tài liệu hướng dẫn, giúp dễ dàng lập trình và mở rộng.

Bảng 2: Thông số kỹ thuật của STM32F103C8T6

Thông số	Giá trị / Đặc điểm
Kiến trúc	ARM Cortex-M3 (32-bit)
Tốc độ xung nhịp tối đa	72 MHz
Bộ nhớ Flash	64 KB
RAM (SRAM)	20 KB
Số chân GPIO	37 chân
Điện áp hoạt động	2.0V – 3.6V
Ngoại vi tích hợp	UART, SPI, I ² C, CAN, USB
ADC	12-bit, 16 kênh
Timer / PWM	16-bit
Chuẩn nạp / gỡ lỗi	ST-Link V2 (SWD)
Dạng board phổ biến	Blue Pill

2.4.2. Vi điều khiển ESP32

ESP32 là một vi điều khiển tích hợp Wifi và Bluetooth, phát triển bởi Espressif Systems, nổi bật với hiệu suất mạnh mẽ và tính năng đa dạng. Được trang bị bộ vi xử lý lõi kép Xtensa LX6, ESP32 chạy ở tần số lên đến 240 MHz, cùng với bộ nhớ RAM lớn và nhiều giao diện kết nối như SPI, I2C, UART, và ADC. Với khả năng kết nối Wi-Fi 802.11 b/g/n và Bluetooth 4.2/BLE, ESP32 cho phép truyền dữ liệu không dây hiệu quả và ổn định. Mạch Wifi này là sự lựa chọn hàng đầu trong các nghiên cứu, ứng dụng về Wi-Fi, BLE, IoT và điều khiển, thu thập dữ liệu qua mạng.

Đặc điểm

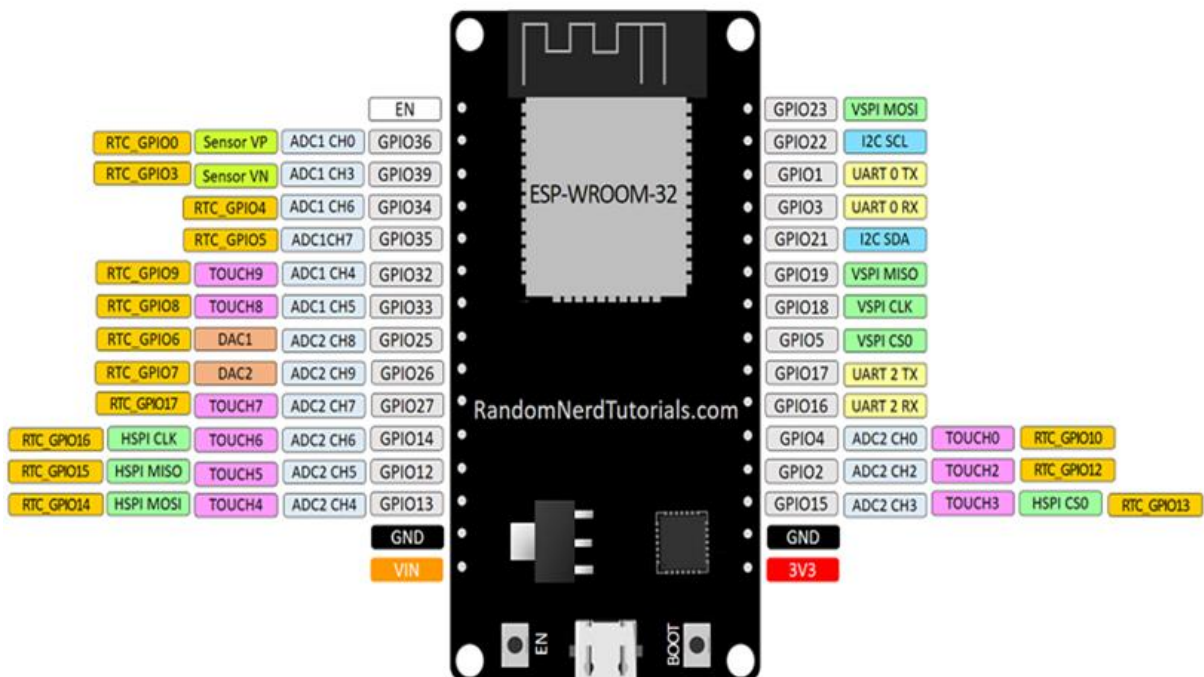
- Kích thước nhỏ gọn, dễ dàng tích hợp vào các sản phẩm khác
- Tính năng mạnh mẽ với giao thức LWIP và Freertos
- Hỗ trợ ba chế độ hoạt động: AP, STA và AP + STA
- Hỗ trợ tất cả các loại giao tiếp:
- 2 bộ chuyển đổi tín hiệu số sang tương tự (DAC) 8 bit
- 2 cổng giao tiếp I2C
- 3 cổng giao tiếp UART
- SD card/SDIO/MMC host.

Ngoài còn có một số ứng dụng ngoại vi: Một số module ESP32 hỗ trợ giao diện máy ảnh, giúp dễ dàng tích hợp vào các ứng dụng camera. Một số module ESP32 còn đi kèm với chân đầu vào cảm ứng.

Bảng 3: Thông số kỹ thuật của ESP32

STT	Chức năng	Mô tả
1	CPU	Xtensa Dual-Core LX6 microprocessor
2	Tốc độ xử lý	160MHZ lên tới 240MHz
3	Tốc độ xung nhịp	40Mhz lên tới 80Mhz
4	RAM	520 KByte SRAM
5	WIFI	802.11 b/g/n/e/i
6	Bluetooth	v4.2 BR/EDR and BLE
7	Nhiệt độ làm việc	-40 + 85C
8	Điện áp làm việc	2.2-3.6V
9	Số chân GPIOs	34

(Nguồn: Datasheet ESP32)



Hình 3: Các thông số chân ESP32

(Nguồn: RandomNerdTutorials.com)

2.4.3. Còi báo (Buzzer)

Còi báo (Buzzer) là thiết bị phát âm thanh dùng để cảnh báo khi hệ thống phát hiện cháy. Trong mô hình, buzzer được điều khiển bởi chân GPIO của ESP32, hoạt động ở mức điện áp 3.3V hoặc 5V. Khi tín hiệu cảm biến IR và nhiệt đều vượt ngưỡng, vi điều khiển xuất tín hiệu kích hoạt còi phát ra âm thanh cảnh báo.

Buzzer được chia thành hai loại: chủ động (tự phát âm khi cấp nguồn) và thụ động (cần xung PWM để tạo âm). Hệ thống sử dụng buzzer chủ động do dễ điều khiển và ổn định.

Đặc điểm:

Âm thanh lớn, dễ nhận biết (khoảng 85 dB).

Dễ tích hợp, tiêu thụ dòng nhỏ.

Hoạt động ổn định trong môi trường 3.3–5V.

Bảng 4: Thông số kỹ thuật của Buzzer

Thông số	Giá trị
Loại	Buzzer chủ động
Điện áp hoạt động	3.3V – 5V DC
Dòng tiêu thụ	~30 mA
Tần số âm thanh	2 – 4 kHz
Cường độ âm thanh	85 dB $\pm$ 5 dB

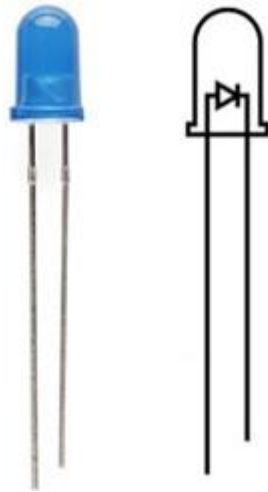


Hình 4: Buzzer

(Nguồn: Nshopvn.com)

2.4.4. Đèn LED cảnh báo

Đèn LED (Light-Emitting Diode) là một thành phần điện tử chuyển đổi điện năng thành ánh sáng. Nó là một nguồn sáng nhỏ gọn, tiết kiệm năng lượng, và có tuổi thọ cao. Đèn LED đã được sử dụng rộng rãi trong nhiều lĩnh vực và có nhiều công dụng quan trọng, trong đó phổ biến nhất là để đưa ra thông báo.



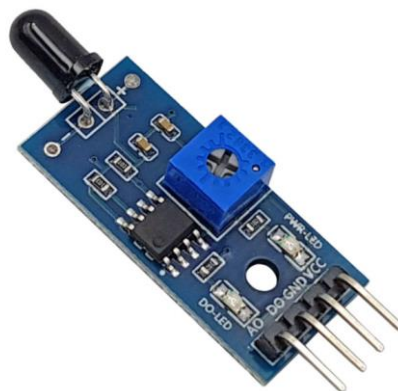
Hình 5: Bóng LED

(Nguồn: forum.arduino.cc)

2.4.5. Cảm biến IR (Flame Sensor)

Cảm biến phát hiện lửa flame sensor thường được sử dụng cho các ứng dụng phát hiện lửa như: xe robot chữa cháy, cảm biến lửa,... Tầm phát hiện trong khoảng 80cm, góc quét là 60 độ, có thể phát hiện lửa tốt nhất là loại có bước sóng từ 760nm – 1100nm.

Cảm biến phát hiện lửa (flame sensor) có hai ngõ ra tín hiệu là Digital và Analog rất dễ sử dụng.



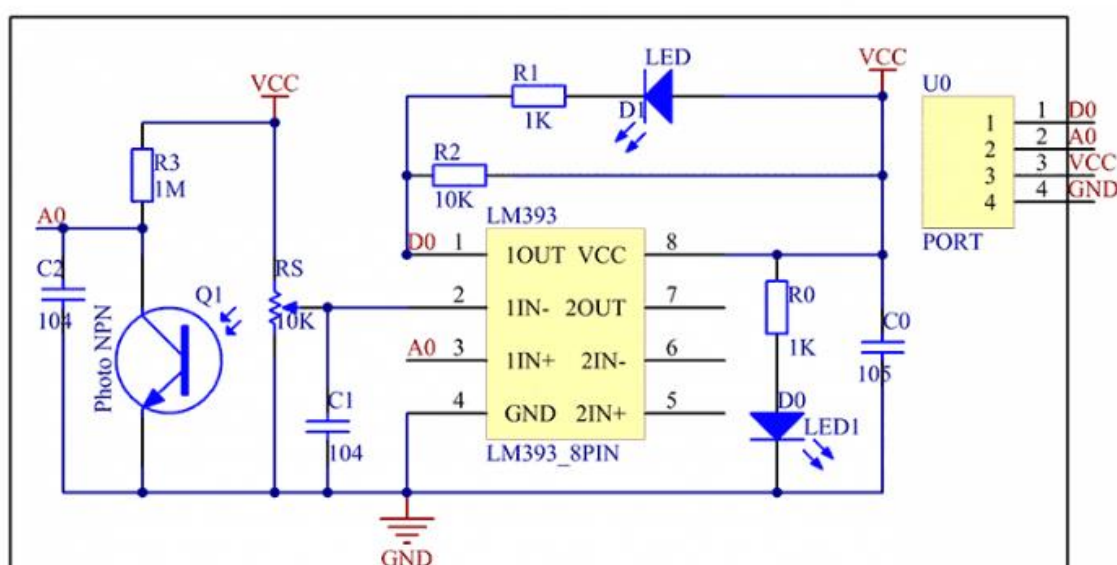
Hình 6: Cảm biến IR (Flame Sensor)

(Nguồn: IC đây rồi)

Bảng 5: Thông số kỹ thuật của IR

STT	Thông số	Giá trị
1	Nguồn cấp	3.3V – 5VDC
2	Dòng tiêu thụ	15mA
3	Tín hiệu ra	Digital 3.3 – 5VDC tùy nguồn cấp hoặc Analog
4	Khoảng cách	80cm
5	Góc quét	60 độ
6	Kích thước	3.2 x 1.4 cm

(Nguồn: IC đây rồi)



Hình 7 : Sơ đồ nguyên lý của cảm biến IR

(Nguồn : nshop.com)

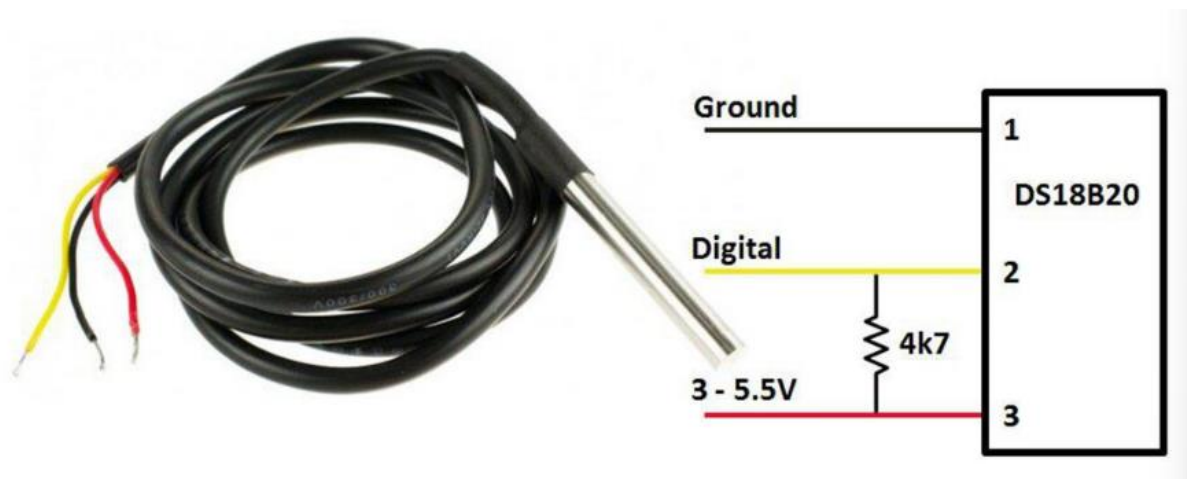
2.4.6. Cảm biến nhiệt độ DS18B20S

Cảm biến nhiệt độ DS18B20 dây mềm, là phiên bản chống nước, chống ẩm của Cảm biến nhiệt độ DS18B20. Cảm biến nhiệt độ DS18B20 là cảm biến (loại digital) đo nhiệt độ mới của hãng MAXIM với độ phân giải cao (12bit). IC sử dụng giao tiếp 1 dây rất gọn gàng, dễ lập trình. IC còn có chức năng cảnh báo nhiệt độ khi vượt ngưỡng và đặc biệt hơn là có thể cấp nguồn từ chân data (parasite power).

Cảm biến nhiệt độ này có thể hoạt động ở 125 độ C nhưng cáp bọc PVC => nên giữ nó dưới 100 độ C. Đây là cảm biến kỹ thuật số, nên không bị suy hao tín hiệu đường dây dài.

Bảng 6 : Thông số của Cảm biến nhiệt độ DS18B20 dây mềm

STT	Thông số	Giá trị
1	Nguồn cấp	3 – 5.5V
2	Dải đo nhiệt độ	-55 đến 125 độ C (-67 đến 257 độ F)
3	Sai số	+/- 0.5 độ C khi đo ở dải -10 – 85 độ C
4	Độ phân giải	người dùng có thể chọn từ 9 – 12 bits
5	Chuẩn giao tiếp	1-Wire (1 dây)
6	Thời gian chuyển đổi nhiệt độ tối đa	750ms (khi chọn độ phân giải 12bit).
7	Đường kính đầu dò	6mm
8	Chiều dài dây	1m



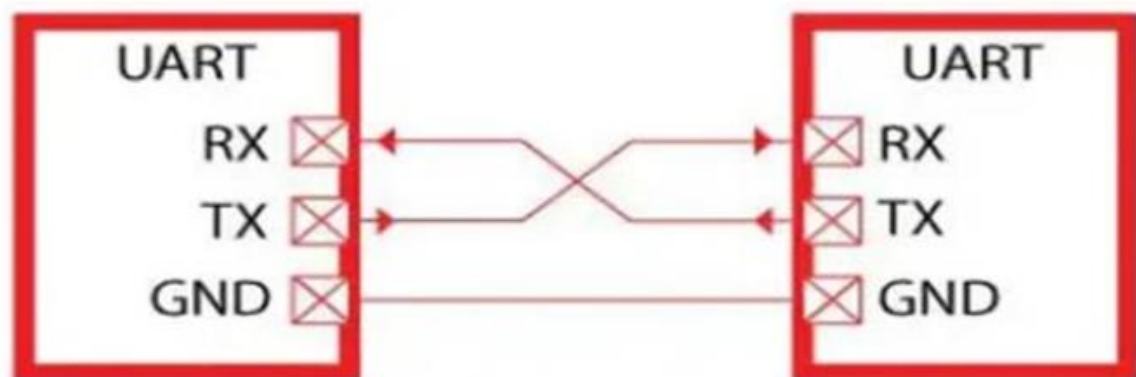
Hình 8 : Sơ đồ kết nối của Cảm biến nhiệt độ DS18B20 dây mềm

(Nguồn: nshop.com)

2.5. Các chuẩn truyền dữ liệu

2.5.1. Chuẩn giao tiếp UART (giữa STM32 và ESP32)

UART (Universal Asynchronous Receiver – Transmitter) là chuẩn giao tiếp nối tiếp không đồng bộ, được sử dụng rộng rãi trong các hệ thống nhúng để truyền dữ liệu giữa hai thiết bị vi điều khiển. UART thực hiện việc chuyển đổi dữ liệu song song trong vi điều khiển thành dữ liệu nối tiếp để truyền qua đường truyền vật lý, và ngược lại. Trong hệ thống này, UART đóng vai trò là cầu nối giữa STM32 và ESP32, cho phép hai vi điều khiển trao đổi dữ liệu dạng chuỗi (byte hoặc ký tự) một cách tin cậy mà không cần xung đồng hồ chung.



Hình 9: Giao tiếp theo chuẩn UART

(Nguồn: Internet)

Nguyên lý hoạt động

Quá trình truyền và nhận được thực hiện thông qua hai chân tín hiệu riêng biệt:

- TX (Transmit): Chân truyền dữ liệu đi.
- RX (Receive): Chân nhận dữ liệu đến.
- Chân TX của thiết bị này sẽ được nối trực tiếp với RX của thiết bị còn lại, và ngược lại. Việc truyền dữ liệu thường diễn ra ở mức điện áp 3.3V hoặc 5V, tùy thuộc vào phần cứng sử dụng.

UART hỗ trợ ba chế độ truyền dữ liệu:

- Simplex: Giao tiếp một chiều, dữ liệu chỉ truyền từ thiết bị phát sang thiết bị thu.
- Half-Duplex: Dữ liệu có thể truyền theo cả hai hướng nhưng chỉ một hướng tại một thời điểm.
- Full-Duplex: Cho phép truyền và nhận dữ liệu đồng thời giữa hai thiết bị.

Về cơ chế hoạt động, dữ liệu trong vi điều khiển được xử lý song song trong CPU, nhưng khi truyền qua UART, nó được chuyển đổi thành dạng nối tiếp để gửi từng bit một qua đường TX. Thiết bị nhận (UART thứ hai) sẽ nhận dữ liệu nối tiếp qua RX, sau đó chuyển lại thành dữ liệu song song để vi điều khiển xử lý.

Dữ liệu truyền qua UART được tổ chức thành các gói (data frame), mỗi gói bao gồm:

- Bit bắt đầu (Start bit): Báo hiệu bắt đầu truyền, kéo mức tín hiệu từ cao xuống thấp.
- 5–9 bit dữ liệu: Thông tin chính được truyền (thường là 8 bit).
- Bit chẵn lẻ (Parity bit – tùy chọn): Dùng để kiểm tra lỗi đơn giản trong quá trình truyền.
- 1–2 bit dừng (Stop bit): Kết thúc khung truyền, đưa tín hiệu trở lại mức cao.
- Tốc độ truyền của UART được xác định bằng tham số baud rate, ví dụ 9600, 115200 bps,...

Khoảng sai lệch tốc độ giữa hai thiết bị không nên vượt quá 10%, nếu không sẽ gây lỗi trong việc đồng bộ thời gian của các bit.

Ưu điểm và nhược điểm của UART

Ưu điểm:

- Cấu trúc đơn giản, dễ lập trình và triển khai.
- Số dây kết nối ít (chỉ cần TX, RX và GND).
- Hỗ trợ truyền song công (full-duplex).
- Phổ biến, có mặt trên hầu hết các vi điều khiển hiện nay.

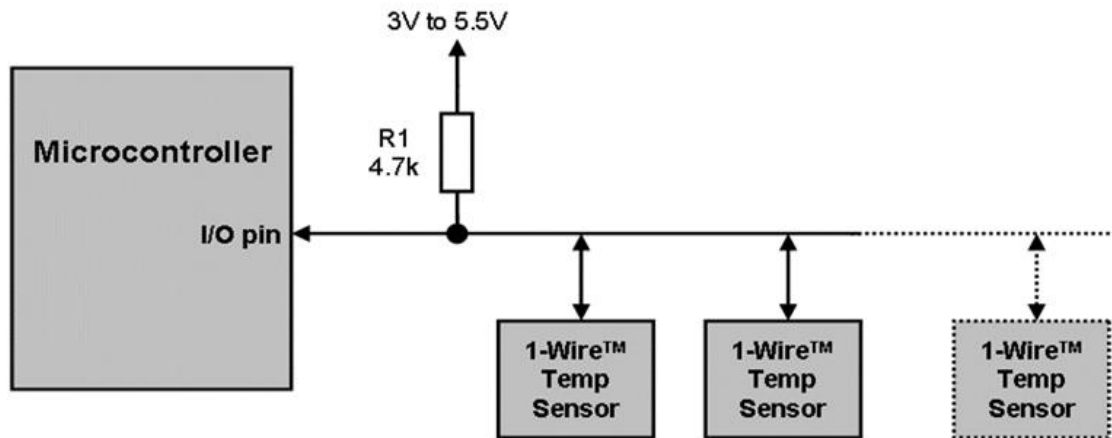
Nhược điểm:

- Tốc độ truyền tương đối thấp so với SPI hoặc CAN.
- Khoảng cách truyền ngắn, dễ nhiễu nếu không che chắn tín hiệu tốt.
- Chỉ hỗ trợ truyền giữa hai thiết bị trong một kênh UART.
- Không thích hợp cho hệ thống có nhiều thiết bị cần đồng bộ dữ liệu.

Ứng dụng: UART thường được sử dụng trong các ứng dụng giao tiếp giữa vi điều khiển và máy tính, module GPS, module Bluetooth, module Wifi (ESP8266, ESP32), hoặc các cảm biến nối tiếp.

2.5.2. Chuẩn giao tiếp 1-Wire

Chuẩn giao tiếp một dây (One Wire) được giới thiệu bởi hãng Dallas là một chuẩn giao tiếp không đồng bộ và bán song công (half-duplex).



Hình 10: Hệ thống truyền dữ liệu bất đồng bộ

(Nguồn: Thegioiic)

Trong chuẩn này, chỉ cần sử dụng một dây để truyền tín hiệu và cung cấp nguồn điện cho các thiết bị (không tính dây mass). Giao tiếp này tuân theo mô hình liên hệ chủ tớ một cách chặt chẽ, trong đó có một master chỉ có thể kết nối với một bus dữ liệu chứa các thiết bị slave. Bus dữ liệu khi không có dữ liệu trên đường truyền phải được kéo lên mức cao, thông qua một điện trở. Giá trị của điện trở này thường được quy định trong datasheet của các thiết bị slave.

Trên một đường truyền dữ liệu có thể gắn nhiều thiết bị slave, nhưng chỉ có duy nhất một thiết bị là master. Mỗi một thiết bị slave sẽ có duy nhất 64-bit địa chỉ lưu trữ trong bộ nhớ ROM của mình. Nhờ thế, khi kết nối vào chung một bus dữ liệu thì thiết bị master sẽ có thể nhận biết được giữa các slave. Trong 8 byte (64 bit) này sẽ được chia ra làm 3 phần chính:

Bắt đầu là LSB (least significant bit), byte đầu tiên bao gồm 8 bit được gửi đi là mã họ thiết bị (family codes) giúp xác định đó là loại thiết bị nào.

6 byte tiếp theo lưu trữ một địa chỉ riêng của từng thiết bị với 48 bit tùy biến.

Byte cuối cùng MSB (most significant bit) là byte kiểm tra tính toàn vẹn của dữ liệu – cyclic redundancy check (CRC), đây là byte giúp kiểm tra xem tín hiệu gửi đi có bị lỗi hay không.

Chuẩn One Wire sử dụng khái niệm time slot (khe thời gian) để truyền và nhận tín hiệu. Mỗi time slot xác định mức logic 0 hoặc 1, có độ dài 60 μs ở chế độ standard và 8 μs ở chế độ overdrive.

Giao tiếp One Wire gồm ba thao tác chính: Write, Read và Reset/Present.

Write:

- Gửi bit 1: Master kéo đường truyền xuống thấp 1–15 μs , rồi nhả lên cao.
- Gửi bit 0: Master giữ mức thấp 60–120 μs , rồi nhả lên cao.
- Giữa các bit có thời gian nghỉ $\geq 1 \mu\text{s}$.

Read:

- Master kéo tín hiệu xuống thấp trong 0–15 μs .
- Slave gửi bit 1: thả đường truyền về mức cao.
- Slave gửi bit 0: giữ mức thấp 15–60 μs .

Reset/Present:

- Master kéo tín hiệu xuống thấp 480–640 μs để reset.
- Sau đó, nếu có thiết bị slave, nó sẽ phản hồi tín hiệu present bằng cách kéo thấp 60–240 μs .
- Nếu không có phản hồi, master hiểu rằng không có slave kết nối.
- Nguyên lý hoạt động
- Master gửi lệnh reset để dò thiết bị slave.
- Khi có phản hồi, master chọn 1 hoặc nhiều slave dựa trên địa chỉ.
- Sau khi xác định slave làm việc, các thiết bị khác bị bỏ qua.
- Master có thể gửi lệnh hoặc đọc dữ liệu từ slave đã chọn.
- Muốn làm việc với slave khác, master phải khởi động lại quy trình từ đầu.

Mặc dù mỗi loại thiết bị có giao thức riêng, nhưng tất cả đều tuân theo quy trình chọn và truyền thông nhất của chuẩn One Wire.

2.5.4. Chuẩn truyền Wifi

Wi-Fi (Wireless Fidelity) là công nghệ truyền thông không dây dựa trên tiêu chuẩn IEEE 802.11, cho phép các thiết bị điện tử kết nối và trao đổi dữ liệu qua sóng vô tuyến mà không cần dây cáp. Công nghệ này hoạt động chủ yếu ở hai dải tần 2.4 GHz và 5GHz, mang lại tốc độ truyền dữ liệu cao và phạm vi phủ sóng khá rộng. Trong hệ thống của đề tài, module ESP32 sử dụng Wi-Fi chuẩn 802.11 b/g/n để truyền dữ liệu cảm biến nhận được từ STM32 lên máy chủ hoặc MQTT Broker, phục vụ cho việc giám sát từ xa.

Về nguyên lý hoạt động, Wifi hoạt động dựa trên cơ chế truyền – nhận sóng vô tuyến giữa hai thành phần chính là Access Point (AP) và Station (STA). Thiết bị AP đóng vai trò phát sóng và điều phối mạng, trong khi thiết bị STA (như ESP32) kết nối và truyền dữ liệu thông qua các gói tin được mã hóa bằng kỹ thuật OFDM hoặc DSSS để đảm bảo tính toàn vẹn và bảo mật của thông tin. Các gói dữ liệu này được truyền qua không khí rồi được giải mã ở thiết bị nhận nhằm khôi phục lại thông tin ban đầu.

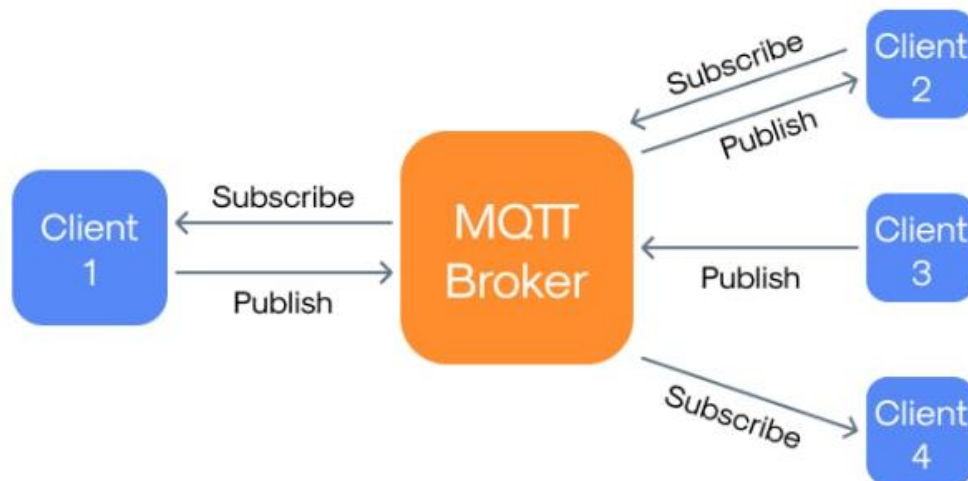
Wi-Fi có một số đặc điểm nổi bật như: phạm vi phủ sóng từ 20 đến 50 mét (có thể mở rộng bằng bộ khuếch đại), tốc độ truyền dữ liệu đạt từ vài chục đến hàng trăm Mb/s tùy theo chuẩn, hỗ trợ nhiều thiết bị truy cập đồng thời, và sử dụng các cơ chế mã hóa bảo mật như WEP, WPA, WPA2, WPA3. Ưu điểm của Wifi là dễ triển khai, tốc độ truyền cao, kết nối linh hoạt và rất phù hợp cho các ứng dụng Internet of Things (IoT). Tuy nhiên, công nghệ này cũng tồn tại hạn chế như dễ bị nhiễu sóng, phạm vi truyền hạn chế, tiêu thụ năng lượng cao và đòi hỏi bảo mật nghiêm ngặt.

Nhờ các đặc điểm trên, Wifi được ứng dụng rộng rãi trong nhà thông minh, hệ thống IoT, giám sát an toàn cháy nổ, và truyền dữ liệu cảm biến. Trong phạm vi đề tài này, việc sử dụng Wi-Fi giúp hệ thống có khả năng gửi cảnh báo cháy và dữ liệu cảm biến theo thời gian thực, giúp người dùng theo dõi và xử lý kịp thời các tình huống nguy hiểm.

2.6. Giao thức MQTT

MQTT (Message Queuing Telemetry Transport) là giao thức truyền thông dạng nhẹ được thiết kế cho các hệ thống IoT có băng thông thấp và độ trễ nhỏ.

Giao thức này hoạt động theo mô hình xuất bản – đăng ký (Publish/Subscribe), trong đó các thiết bị trao đổi dữ liệu thông qua một máy chủ trung gian gọi là Broker.



Hình 11: Giao tiếp với giao thức MQTT

MQTT gồm ba thành phần chính:

Publisher: Thiết bị hoặc ứng dụng gửi thông điệp (message) đến một chủ đề (topic) cụ thể.

Subscriber: Thiết bị hoặc ứng dụng đăng ký nhận thông điệp từ một hoặc nhiều chủ đề mà nó quan tâm.

Broker: Chịu trách nhiệm nhận các thông điệp từ publisher và phân phối chúng đến các subscriber tương ứng. Broker cũng quản lý việc kết nối, bảo mật và đảm bảo chất lượng truyền tải.

Lý do chọn MQTT trong hệ thống IoT cho phát hiện lửa và cảm biến nhiệt

- Đảm bảo độ tin cậy cao: Hỗ trợ các mức QoS (Quality of Service) 0, 1, 2 cho phép lựa chọn giữa tốc độ và độ đảm bảo truyền dữ liệu. Có cơ chế Last Will Message và KeepAlive để xử lý mất kết nối.

- Hỗ trợ bảo mật bằng SSL/TLS: MQTT có thể hoạt động trên kênh bảo mật (port 8883) với chứng chỉ SSL/TLS, giúp mã hóa dữ liệu, bảo vệ thông tin khi truyền qua Internet. Điều này phù hợp với yêu cầu an toàn thông tin trong hệ thống IoT.

- Thích hợp cho ứng dụng thời gian thực: MQTT có độ trễ thấp, điều đó giúp hỗ trợ cập nhật dữ liệu gần như tức thì giữa thiết bị và máy chủ.

- Dễ dàng tích hợp với các nền tảng web và cloud bao gồm: HiveMQ, Mosquitto, EMQX, AWS IoT, Adafruit IO, v.v.

Mức chất lượng dịch vụ (Quality of Service – QoS)

MQTT có 3 mức QoS (0, 1, 2) mỗi mức thể hiện độ đảm bảo dữ liệu khác nhau giữa client và broker:

QoS 0: At most once (Tối đa một lần)

- Gửi tối đa một lần.
- Không đảm bảo đến nơi, có thể bị mất.
- Nhanh và ít tốn tài nguyên.

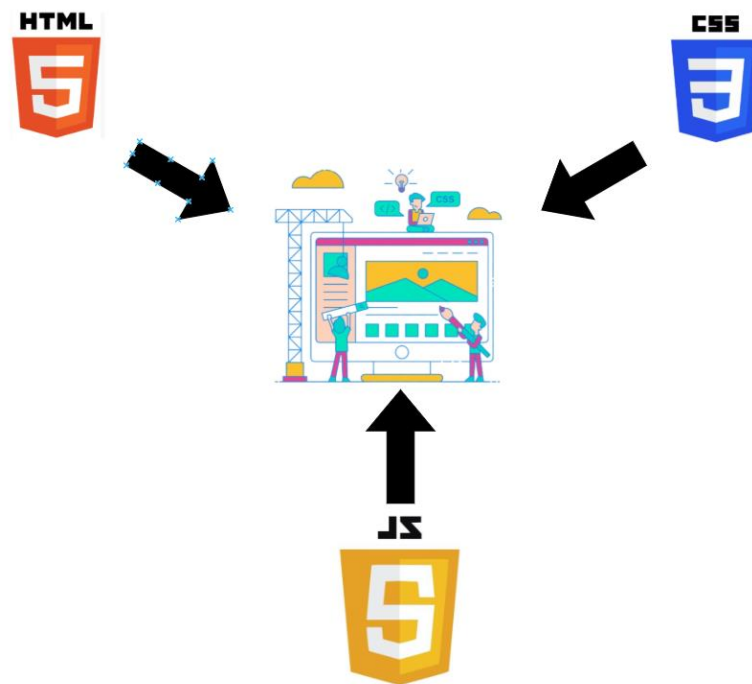
QoS 1: At least once (Ít nhất một lần)

- Đảm bảo thông điệp đến.
- Có thể nhận trùng lặp.

QoS 2: Exactly once (Chính xác một lần)

- Đảm bảo chính xác một lần.
- An toàn nhất nhưng chậm hơn và tốn nhiều tài nguyên hơn.

2.7. UI (User interface) và UX (User experience)



Hình 12: Cấu trúc kết hợp HTML – CSS – JavaScript trong phát triển giao diện web

2.7.1. HTML

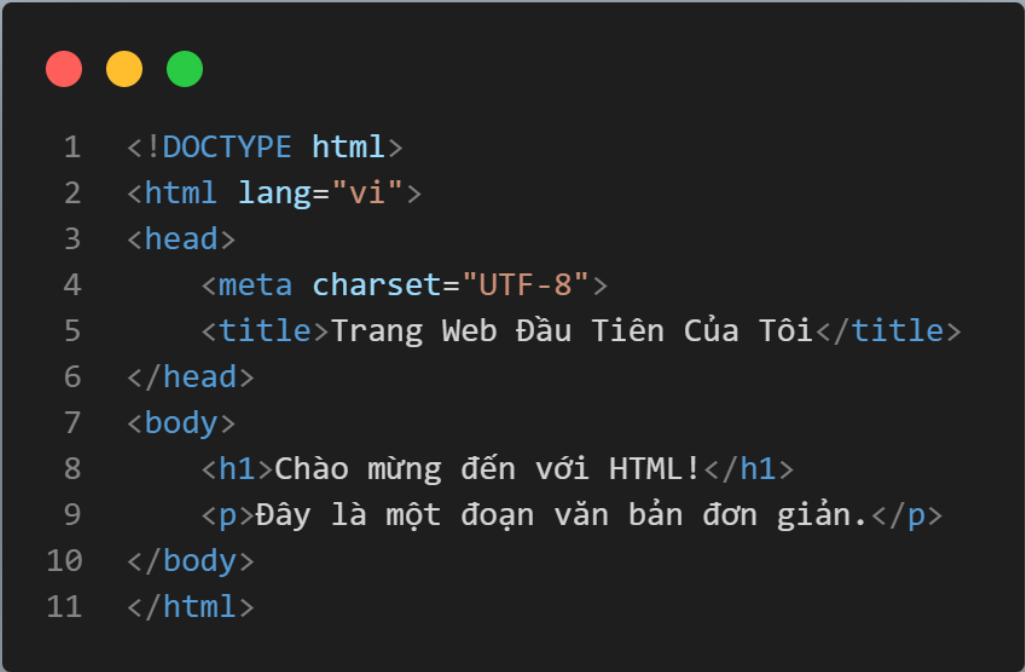
HTML là viết tắt của cụm từ Hypertext Markup Language (tạm dịch là Ngôn ngữ đánh dấu siêu văn bản). HTML được sử dụng để tạo và cấu trúc các thành phần trong trang web hoặc ứng dụng, phân chia các đoạn văn, heading, titles, blockquotes... và HTML không phải là ngôn ngữ lập trình.



Hình 13: Logo HTML

Cấu trúc cơ bản của một trang HTML được tổ chức thành ba phần chính. Thứ nhất là phần Khai báo Loại File Code với cú pháp `<!DOCTYPE html>`, có nhiệm vụ thông báo cho trình duyệt về phiên bản HTML đang được sử dụng. Thứ hai là phần Khai báo Ban đầu, được định nghĩa bởi cặp thẻ `<head>` và `</head>`. Phần này chứa các thông tin metadata quan trọng nhằm hỗ trợ SEO, bao gồm tiêu đề trang (đặt trong cặp thẻ `<title>`), các khai báo về meta, liên kết tới CSS và JavaScript. Cuối cùng và quan trọng nhất là phần Nội dung Trang web, được đặt trong cặp thẻ `<body>` và `</body>`. Toàn bộ nội dung hiển thị cho người dùng—bao gồm tiêu đề trang, logo, các thanh điều hướng, nội dung chính của bài viết, thanh bên (sidebar) và chân trang (footer) cung cấp thông tin liên hệ và bản quyền—đều được định nghĩa và chứa đựng trong phần tử `<body>` này.

Ví dụ minh họa:

A code editor window with a dark background and three colored window control buttons (red, yellow, green) in the top-left corner. The editor contains 11 lines of HTML code, with line numbers 1 through 11 on the left. The code is as follows:

```
1 <!DOCTYPE html>
2 <html lang="vi">
3 <head>
4     <meta charset="UTF-8">
5     <title>Trang Web Đầu Tiên Của Tôi</title>
6 </head>
7 <body>
8     <h1>Chào mừng đến với HTML!</h1>
9     <p>Đây là một đoạn văn bản đơn giản.</p>
10 </body>
11 </html>
```

2.7.2. CSS

CSS là viết tắt của Cascading Style Sheets, nó là một ngôn ngữ được sử dụng để tìm và định dạng lại các phần tử được tạo ra bởi các ngôn ngữ đánh dấu (HTML).



Hình 14: Logo CSS

Ngôn ngữ CSS hoạt động dựa trên các Quy tắc kiểu dáng (Style Rules) có cấu trúc rõ ràng gồm có bộ chọn (selector) để chỉ định phần tử HTML cần được định dạng, khai báo (declaration) gồm cặp thuộc tính (property) và giá trị (value) tương ứng, được phân tách bằng dấu hai chấm (:), và luôn kết thúc bằng dấu chấm phẩy (;) để phân tách các khai báo.

Ví dụ minh họa:

```
1  <!DOCTYPE html>
2  <html lang="vi">
3  <head>
4      <meta charset="UTF-8">
5      <title>Ví Dụ CSS</title>
6      <link rel="stylesheet" href="style.css">
7  </head>
8  <body>
9
10     <header>
11         <h1>Đây là Tiêu đề Chính</h1>
12     </header>
13
14     <div class="content">
15         <p id="first-paragraph">Đoạn văn này có ID riêng.</p>
16         <p>Đoạn văn này sẽ có kiểu dáng khác.</p>
17     </div>
18
19     <button class="primary-btn">Nhấn vào đây</button>
20
21 </body>
22 </html>
```


2.7.3. Javascript (JS)

JavaScript (JS) là ngôn ngữ lập trình được sử dụng để tạo ra tính tương tác và động cho các trang web.



Hình 15: Logo Javascript

Trong bộ ba công nghệ nền tảng của web, nếu HTML định hình cấu trúc nội dung và CSS quy định giao diện (cách hiển thị), thì JavaScript là ngôn ngữ đảm nhận việc xử lý hành vi, sự kiện và logic động trên trình duyệt. Nhờ JavaScript, trang web trở nên linh hoạt, tương tác và có khả năng phản hồi theo hành động của người dùng (như nhấn nút, di chuột, nhập dữ liệu).

JavaScript có thể được tích hợp vào tài liệu HTML bằng cách đặt mã trực tiếp bên trong thẻ `<script>` hoặc được lưu trữ trong một tệp riêng biệt có phần mở rộng `.js` để dễ dàng quản lý và bảo trì.

Ví dụ minh họa:

```

1  <!DOCTYPE html>
2  <html lang="vi">
3  <head>
4      <meta charset="UTF-8">
5      <title>Ví Dụ JavaScript</title>
6  </head>
7  <body>
8
9      <h1 id="message">Trạng thái ban đầu: Chưa nhấn nút</h1>
10
11     <button id="actionButton">Nhấn vào đây để thay đổi</button>
12
13     <script>
14         // 1. Khai báo hàm xử lý sự kiện
15         function changeMessage() {
16             // Lấy phần tử H1 bằng ID của nó
17             const messageElement = document.getElementById('message');
18
19             // Lấy phần tử nút bấm
20             const buttonElement = document.getElementById('actionButton');
21
22             // 2. Thay đổi nội dung (logic)
23             if (messageElement.textContent === 'Trạng thái ban đầu: Chưa nhấn nút') {
24                 messageElement.textContent = '✅ Nội dung đã được thay đổi bằng JavaScript!';
25                 messageElement.style.color = 'green'; // Thêm hiệu ứng CSS
26                 buttonElement.textContent = 'Khôi phục Trạng thái';
27             } else {
28                 messageElement.textContent = 'Trạng thái ban đầu: Chưa nhấn nút';
29                 messageElement.style.color = 'black'; // Khôi phục màu
30                 buttonElement.textContent = 'Nhấn vào đây để thay đổi';
31             }
32         }
33
34         // 3. Gán hàm xử lý sự kiện (Hành vi)
35         // Khi nút có ID là 'actionButton' được click, hàm changeMessage sẽ được gọi
36         document.getElementById('actionButton').addEventListener('click', changeMessage);
37     </script>
38
39 </body>
40 </html>

```

CHƯƠNG 3: THIẾT KẾ VÀ THI CÔNG MÔ HÌNH

3.1. Phân tích yêu cầu hệ thống

Hệ thống phát hiện ngọn lửa bằng cảm biến hồng ngoại kết hợp xác nhận bằng nhiệt độ được thiết kế nhằm đạt hai mục tiêu chính là phát hiện cháy sớm và giảm thiểu cảnh báo sai. Để đáp ứng yêu cầu này, hệ thống phải bảo đảm các tiêu chí kỹ thuật về độ chính xác, tốc độ phản hồi, tính ổn định của truyền thông và khả năng mở rộng trong môi trường ứng dụng thực tế.

Về yêu cầu chức năng, hệ thống cần phát hiện sự xuất hiện của ngọn lửa thật thông qua cảm biến hồng ngoại (IR Flame Sensor). Khi cảm biến phát hiện bức xạ hồng ngoại vượt ngưỡng, tín hiệu được gửi đến vi điều khiển STM32 để xử lý. Ở bước xác nhận thứ hai, vi điều khiển đọc giá trị từ cảm biến nhiệt độ. Nếu nhiệt độ môi trường tăng vượt ngưỡng đã được cài đặt, hệ thống xác định đây là tình huống cháy thực sự và tiến hành kích hoạt cảnh báo.

Về yêu cầu kỹ thuật, thời gian phản hồi của hệ thống từ khi phát hiện ngọn lửa đến khi cảnh báo không vượt quá một giây. Tỷ lệ báo động sai cần được giới hạn dưới 5% trong điều kiện môi trường có ánh sáng mạnh hoặc thay đổi nhiệt độ bất thường. Hệ thống phải hoạt động ổn định trong khoảng nhiệt độ từ 20°C đến 50°C và có khả năng làm việc liên tục nhiều giờ mà không bị lỗi truyền thông.

Về yêu cầu truyền thông, hệ thống cần truyền dữ liệu phát hiện cháy lên máy chủ MQTT Broker trong thời gian thực. Dữ liệu được hiển thị trực quan trên nền tảng Node-RED Dashboard hoặc kiểm tra bằng MQTT Explorer để người dùng có thể theo dõi, lưu trữ và phân tích sau này.

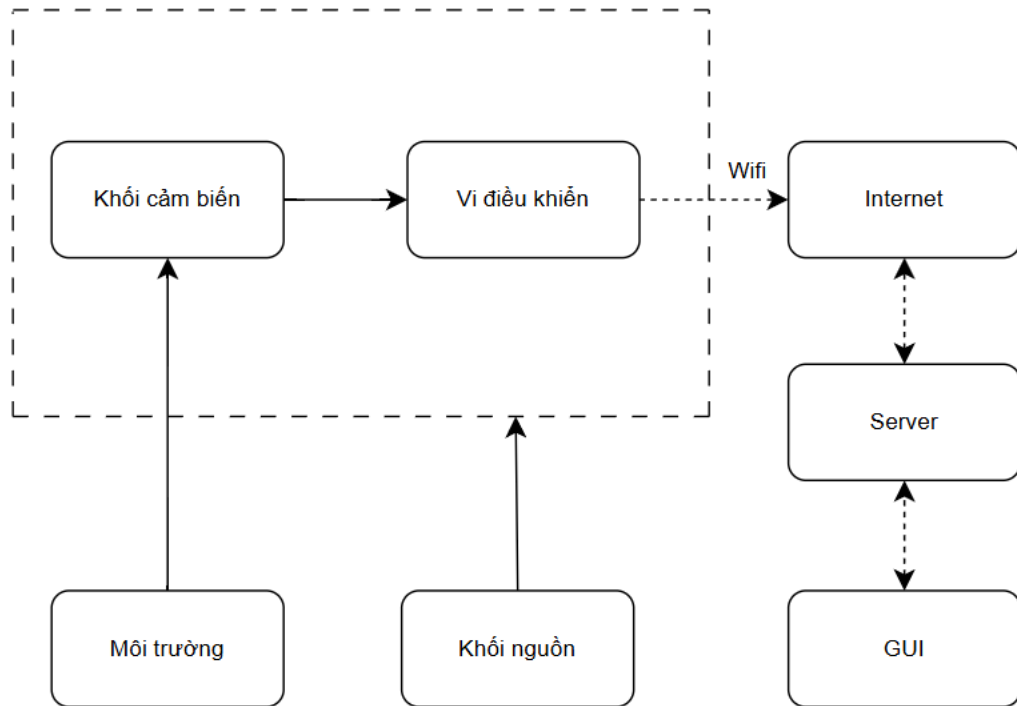
3.2. Sơ đồ khối và nguyên lý hoạt động

Hệ thống được thiết kế theo mô hình phân tầng IoT gồm bốn tầng: tầng cảm nhận, tầng xử lý, tầng truyền thông, và tầng ứng dụng. Các khối phần cứng chính bao gồm STM32, ESP32, cảm biến ngọn lửa (IR), cảm biến nhiệt độ (DS18B20S), LED cảnh báo, và buzzer (còi).

3.2.1. Sơ đồ khối tổng quát

Tổng thể hệ thống có cấu trúc như Hình 3.1. Trong đó, STM32 đóng vai trò là bộ thu thập dữ liệu cảm biến, ESP32 đảm nhiệm việc phân tích – xử lý tín

hiệu – điều khiển thiết bị cảnh báo, đồng thời thực hiện truyền dữ liệu qua MQTT Broker đến giao diện hiển thị web hoặc Node-RED Dashboard.



Hình 16: Sơ đồ khối tổng quát hệ thống phát hiện cháy

Hệ thống được xây dựng bao gồm có 2 khối kết nối với nhau:

- Khối cảm biến: Khối này dung cảm biến đo nhiệt và cảm biến phát hiện lửa, nhằm thực hiện theo dõi quan sát môi trường xung quanh và cảnh báo khi có vấn đề xảy ra.

- Khối vi điều khiển: Khối này là khối trang bị các thuật toán, tương tác với dữ liệu được thu thập từ khối cảm biến và đồng thời chuyển dữ liệu đã xử lý đến lên Internet.

- Khối nguồn: Là khối cấp nguồn 5VDC cho các khối cảm biến và khối xử lý trung tâm.

- Database: Hệ thống sử dụng các database để thuận lợi cho việc giao tiếp giữa các phần cứng với web.

- Web: Là một giao diện được lập trình bởi HTML, JS và CSS; mục đích làm web để giúp thu thập dữ liệu, các thông tin liên quan tới môi trường. Dựa vào đó mà

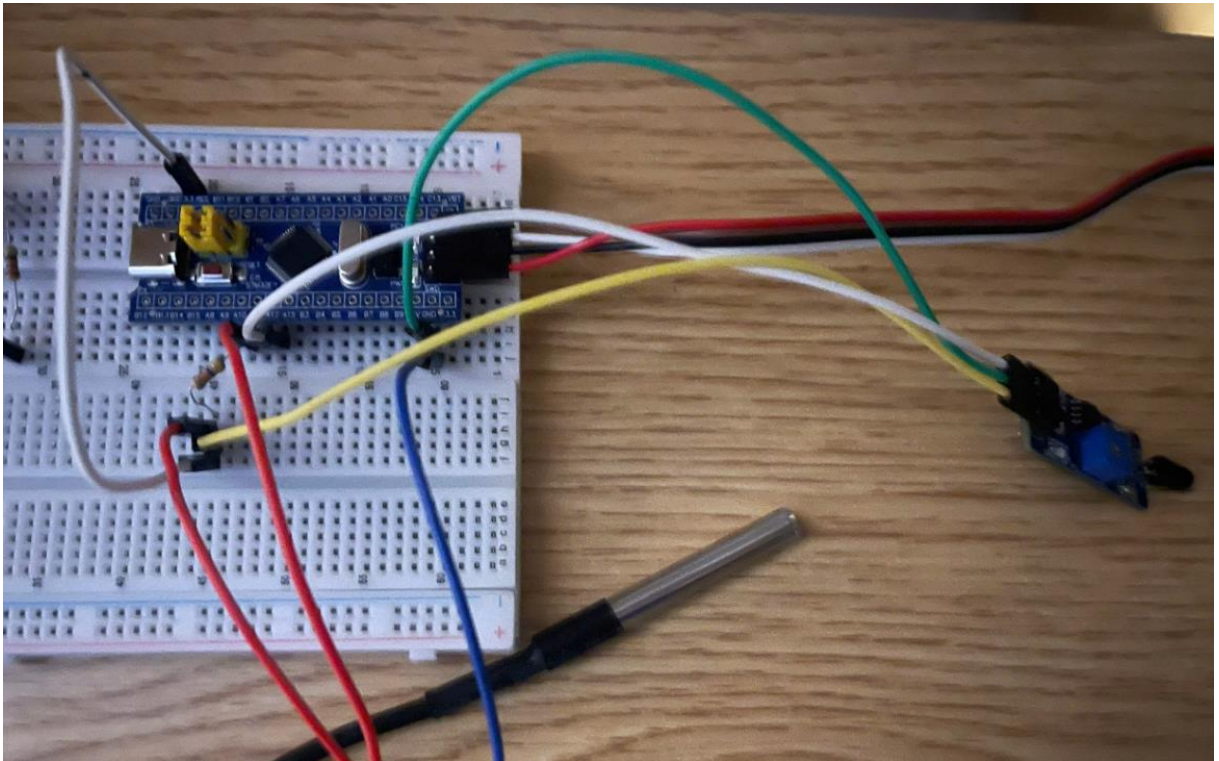
3.2.2. Tầng cảm nhận

Tầng cảm nhận bao gồm hai cảm biến:

Cảm biến hồng ngoại (IR Flame Sensor) phát hiện bức xạ đặc trưng của ngọn lửa trong dải 760–1100 nm.

Cảm biến nhiệt độ (DS18B20) ghi nhận sự biến thiên nhiệt độ môi trường để xác nhận tín hiệu từ cảm biến IR.

Cả hai cảm biến được kết nối trực tiếp đến STM32 thông qua các chân ADC hoặc 1-Wire. Tín hiệu được lấy mẫu theo chu kỳ định sẵn (khoảng 100 ms) nhằm đảm bảo dữ liệu thời gian thực.



Hình 17: Sơ đồ kết nối phần cảm biến với STM32

3.2.3. Tầng xử lý trung gian (STM32)

Vi điều khiển STM32 thực hiện các nhiệm vụ:

Đọc dữ liệu cảm biến thông qua ADC.

Lọc nhiễu, tính trung bình nhiều mẫu để giảm sai số.

Đóng gói dữ liệu dạng khung UART gồm:

```
<S,ir=XXXX,temp=YY.Y,tick=ZZZZ,cs=AA>\r\n
```

Trong đó: ir là giá trị cảm biến hồng ngoại, temp là nhiệt độ, tick là bộ đếm mẫu và cs là checksum.

Gửi khung dữ liệu này cho ESP32 qua giao tiếp UART ở tốc độ 115200 bps.

STM32 chỉ đảm nhận chức năng thu thập và truyền dữ liệu, không đưa ra quyết định cảnh báo.

3.2.4. Tầng xử lý chính (ESP32)

ESP32 là bộ xử lý trung tâm của hệ thống, có nhiệm vụ:

Nhận chuỗi dữ liệu UART từ STM32, kiểm tra checksum và phân tách giá trị.

So sánh với các ngưỡng được cài đặt (IR_threshold và Temp_threshold).

Thực hiện thuật toán xác nhận hai bước:

Nếu IR vượt ngưỡng, ESP32 bắt đầu theo dõi nhiệt độ.

Nếu nhiệt độ tăng liên tiếp và vượt ngưỡng trong 1 s → xác nhận cháy thật.

Khi xác nhận cháy:

Kích hoạt LED đỏ sáng liên tục.

Bật buzzer trong 3 s, sau đó chuyển sang chế độ nhấp nháy nếu sự kiện kéo dài.

Gửi bản tin MQTT lên Broker để hiển thị cảnh báo trên giao diện.

ESP32 được lập trình bằng Arduino IDE, kết nối Wi-Fi chuẩn 802.11 b/g/n. Nó hoạt động như MQTT Client, publish dữ liệu đến Broker tại topic.

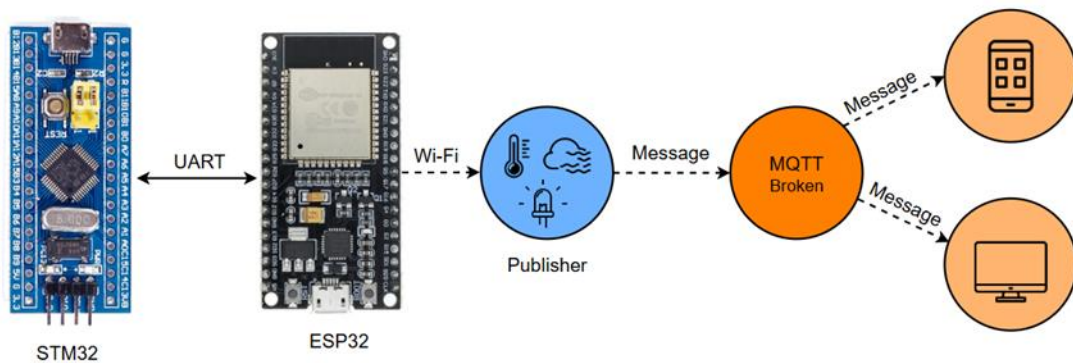
3.2.5. Tầng truyền thông

Giao tiếp giữa các tầng gồm hai phần:

UART (STM32 → ESP32): truyền dữ liệu dạng khung, tốc độ 115200 bps, không đồng bộ, 8N1.

MQTT (ESP32 → Broker): truyền dữ liệu qua Wi-Fi nội bộ, QoS 0, duy trì kết nối với Broker Mosquitto.

ESP32 đóng vai trò publisher, trong khi Dashboard hoặc ứng dụng web là subscriber. Khi mất kết nối Wi-Fi, ESP32 tự động lưu trữ bản tin tạm trong hàng đợi và gửi lại khi mạng ổn định.



Hình 18: Sơ đồ truyền thông giữa STM32, ESP32 và MQTT Broker

3.2.6. Tầng ứng dụng

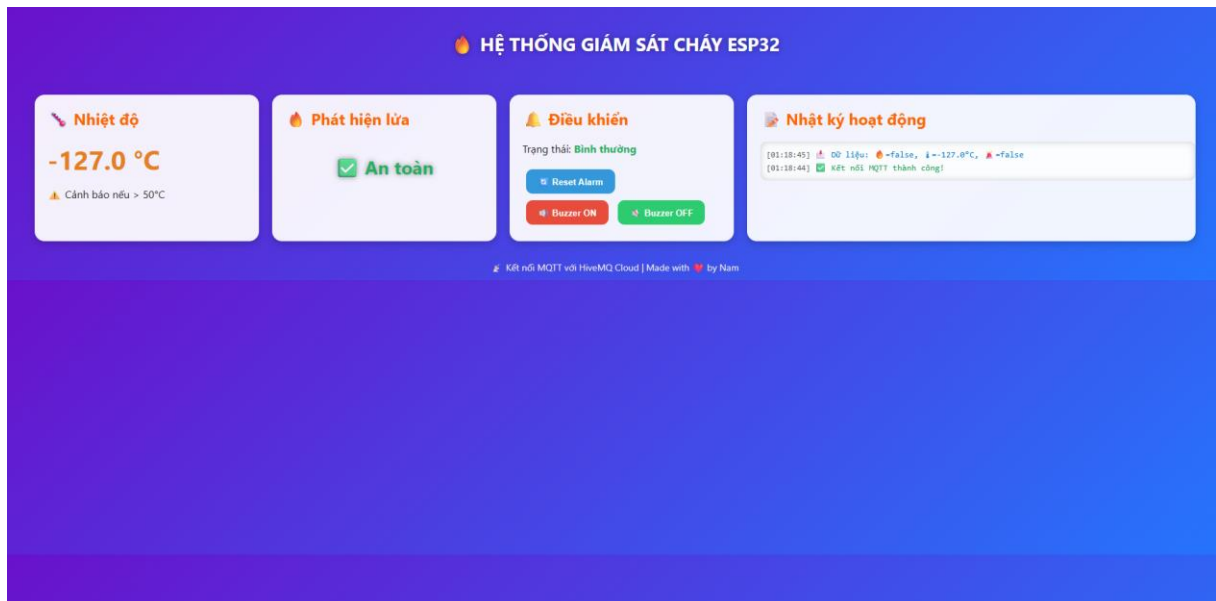
Tầng ứng dụng gồm giao diện Node-RED Dashboard hoặc web app, giúp người dùng quan sát dữ liệu theo thời gian thực. Các thành phần hiển thị:

Đồng hồ hiển thị nhiệt độ.

Biểu đồ thời gian thực của giá trị IR và nhiệt độ.

Cảnh báo bằng màu sắc và thông báo pop-up khi phát hiện cháy.

Lưu nhật ký dữ liệu theo ngày.



Hình 19: Giao diện hiển thị dữ liệu và cảnh báo cháy thời gian thực

3.2.7. Nguyên lý hoạt động tổng quát

Cảm biến IR phát hiện bức xạ hồng ngoại của ngọn lửa → gửi tín hiệu tương tự đến STM32.

Cảm biến nhiệt độ đo giá trị môi trường → STM32 đọc, tính toán, gửi dữ liệu qua UART.

ESP32 nhận dữ liệu, phân tích điều kiện.

Nếu cả hai điều kiện IR & Temp đều vượt ngưỡng → ESP32 bật LED/còi
→ publish bản tin MQTT.

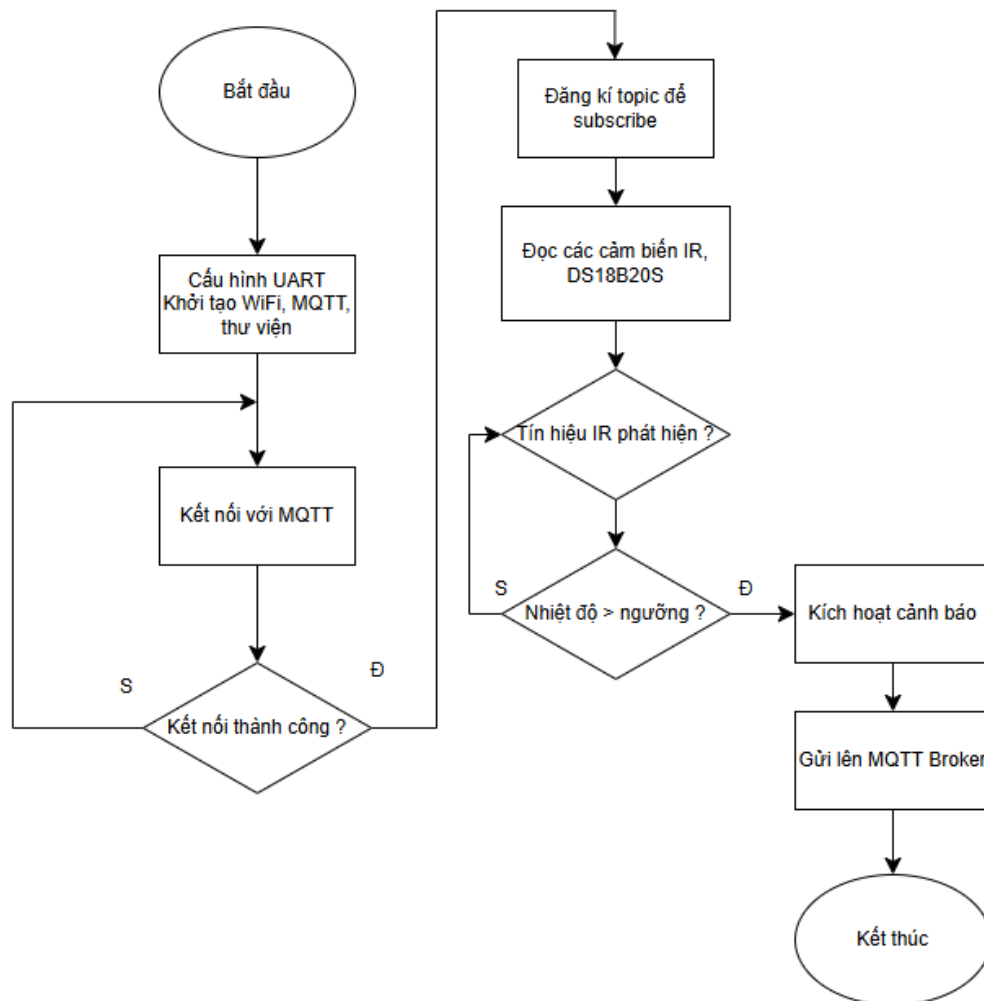
MQTT Broker chuyển bản tin đến Node-RED Dashboard → người dùng nhận cảnh báo.

3.3. Lập trình hệ thống

3.3.1. Yêu cầu của phần mềm

ESP32 là khối xử lý trung tâm, đảm nhận kết nối Wi-Fi, phiên MQTT, xử lý hai điều kiện phát hiện cháy và điều khiển LED/còi. STM32 đảm nhận thu thập dữ liệu cảm biến và truyền cho ESP32 qua UART ở tốc độ baud ổn định (ví dụ 115200 bps). Dữ liệu tối thiểu gồm giá trị IR-ADC, giá trị nhiệt độ đã quy đổi theo °C và dấu thời gian cục bộ của STM32 hoặc chỉ số mẫu liên tục để ESP32 đồng bộ của số tính toán.

3.3.2. Lưu đồ giải thuật chương trình chính của hệ thống



Hình 20: Lưu đồ chương trình chính của hệ thống

Lưu đồ trên mô tả quy trình hoạt động của hệ thống cảnh báo cháy sử dụng vi điều khiển STM32 và module ESP32 trong mô hình IoT. Khi được cấp nguồn, hệ thống khởi tạo các thành phần cần thiết, bao gồm cấu hình UART để truyền dữ liệu giữa STM32 và ESP32, thiết lập kết nối Wi-Fi, khởi tạo giao thức MQTT cùng các thư viện hỗ trợ truyền thông. Sau khi hoàn tất quá trình khởi tạo, ESP32 tiến hành kết nối với máy chủ MQTT Broker. Trong trường hợp kết nối không thành công, hệ thống sẽ tự động thử lại cho đến khi kết nối được thiết lập ổn định.

Khi đã kết nối thành công với MQTT Broker, hệ thống chuyển sang chế độ giám sát. STM32 liên tục thu thập dữ liệu từ hai cảm biến là cảm biến IR và cảm

biến nhiệt độ. Cảm biến IR có nhiệm vụ phát hiện tia hồng ngoại phát ra từ ngọn lửa, trong khi cảm biến nhiệt theo dõi sự thay đổi của nhiệt độ môi trường. Khi cảm biến IR phát hiện tín hiệu nghi ngờ có ngọn lửa, STM32 sẽ gửi dữ liệu sang ESP32 để xác minh thêm bằng cách kiểm tra giá trị nhiệt độ hiện tại. Nếu nhiệt độ vượt quá ngưỡng cài đặt, hệ thống xác nhận có cháy, đồng thời kích hoạt còi báo và đèn LED cảnh báo.

Sau khi cảnh báo được kích hoạt, ESP32 gửi dữ liệu trạng thái và thông báo lên MQTT Broker để hiển thị trên giao diện giám sát như Node-RED Dashboard hoặc trang web dashboard tùy chỉnh. Các thông tin hiển thị bao gồm giá trị nhiệt độ, tín hiệu IR, số lần cảnh báo và trạng thái kết nối của hệ thống. Sau khi hoàn tất quá trình truyền dữ liệu, hệ thống quay trở lại trạng thái giám sát ban đầu để tiếp tục theo dõi, đảm bảo hoạt động liên tục và ổn định.

3.3.3. Khung dữ liệu UART STM32 → ESP32

Để giảm lỗi và thuận tiện debug, sử dụng khung ASCII cố định, có tiền tố, hậu tố và checksum đơn giản. Ví dụ định dạng:

```
<S,ir=0123,temp=026.7,tick=12345,cs=5A>\r\n
```

Trong đó “S” là nguồn STM32, “ir” là giá trị ADC đã chuẩn hóa 0–4095, “temp” là nhiệt độ °C với một chữ số thập phân, “tick” là bộ đếm mẫu, “cs” là checksum dạng XOR 8-bit của toàn chuỗi giữa dấu “<...>”. ESP32 chỉ nhận chuỗi hợp lệ khi checksum khớp, nếu không sẽ bỏ qua và yêu cầu STM32 gửi khung kế tiếp.

3.3.4. Chủ đề (topic) MQTT và payload

Đề xuất namespace theo khu vực triển khai, ví dụ:

- iot/fire/siteA/node01/state cho trạng thái Safe/Alarm,
- iot/fire/siteA/node01/telemetry cho dữ liệu thời gian thực.

Payload dạng JSON tối giản:

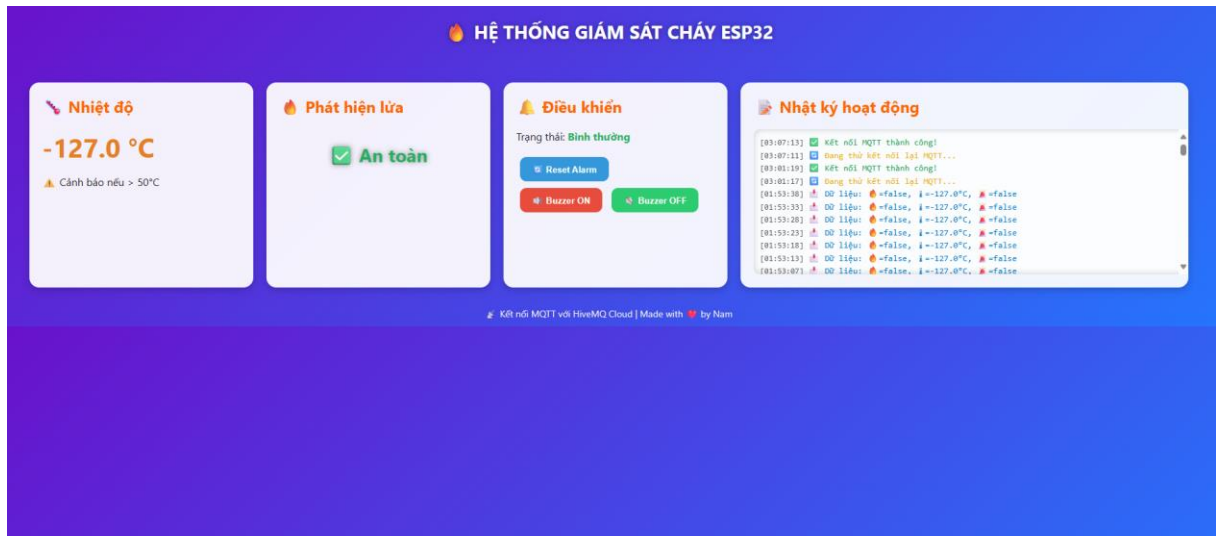
```
{"ts":"2025-10-21T14:32:05Z","ir":123,"temp":26.7,"alarm":true}
```

ESP32 publish với QoS 1 để đảm bảo đến đích; Dashboard/website subscribe và hiển thị ngay.

3.4. Thiết kế giao diện giám sát

3.4.1. Cấu trúc giao diện của người dùng

Toàn bộ giao diện web được xây dựng trên HTML, JS và CSS, mục đích để quan sát nhiệt độ, nhật ký hoạt động và phát hiện ngọn lửa.



Nội dung chính được chia thành 2 phần:

- Hiển thị nhiệt độ của môi trường.
- Phát hiện lửa.

3.4.2. Cấu trúc HTML tổng quát



```
1  <!DOCTYPE html>
2  <html lang="vi">
3  <head> ... </head>
4
5  <body>
6    <h1>...</h1>
7    <div class="container">
8      <!-- Khối nhiệt độ -->
9      <!-- Khối phát hiện lửa -->
10     <!-- Khối điều khiển -->
11     <!-- Nhật ký -->
12   </div>
13   <div class="footer">...</div>
14
15   <script>...</script>
16 </body>
17 </html>
```

3.4.3. Chức năng điều khiển



Giao diện điều khiển cho phép người dung tương tác trực tiếp với hệ thống cảnh báo cháy thông qua các nút chức năng được kết nối qua giao thức MQTT.

- Nút “Reset Alarm”: Gửi lệnh đặt lại hệ thống, tắt còi và đèn cảnh báo sau khi tình huống cháy đã được xử lý.

```
04:00:34.910 -> MQTT msg: {"command":"reset"}
04:00:34.910 -> → Alarm reset
```

- Nút “Buzzer ON”: Bật còi thủ công nhằm kiểm tra hoạt động của hệ thống hoặc cảnh báo khẩn.

```
04:02:37.611 -> MQTT msg: {"buzzer":"on"}
04:02:37.611 -> → Buzzer on
```

- Nút “Buzzer OFF”: Tắt còi khi không còn nguy cơ cháy.

```
04:03:31.055 -> MQTT msg: {"buzzer":"off"}
04:03:31.055 -> → Buzzer off
```

CHƯƠNG 4: THỰC NGHIỆM, KIỂM CHỨNG VÀ ĐÁNH GIÁ HỆ THỐNG

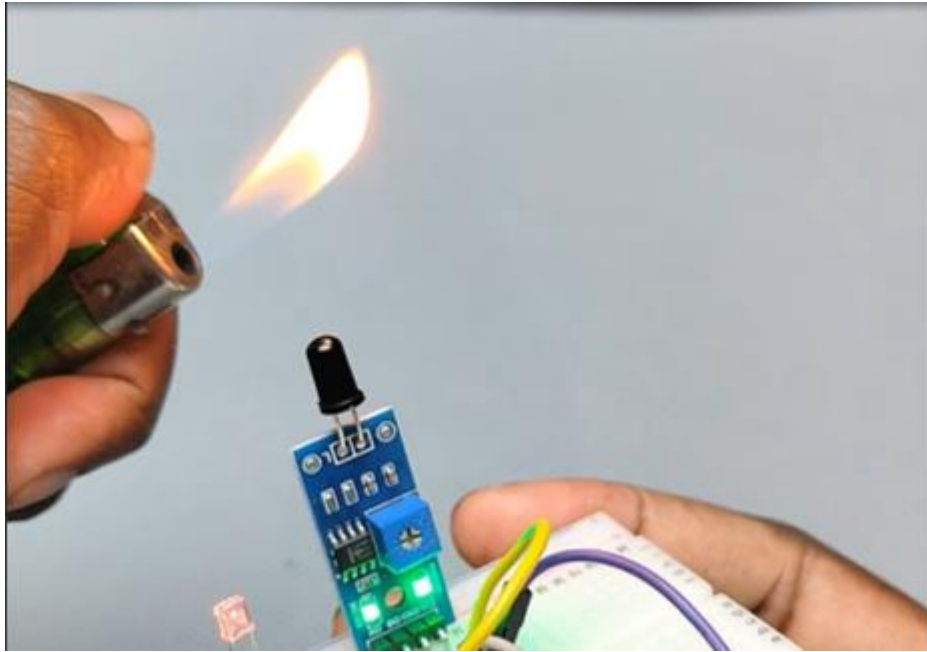
4.1. Mô hình hệ thống thực tế

Hệ thống phát hiện ngọn lửa bằng cảm biến hồng ngoại kết hợp xác nhận bằng nhiệt độ được lắp ráp trên mô hình mạch thực tế bao gồm các thành phần: cảm biến IR Flame, cảm biến nhiệt độ, vi điều khiển STM32F103C8T6, module Wifi ESP8266, còi cảnh báo, đèn LED và nguồn 5V DC. Các linh kiện được bố trí trên breadboard để thuận tiện cho việc thử nghiệm và đo đạc.

Tín hiệu đầu ra từ cảm biến IR được đưa vào chân ADC của STM32 để chuyển đổi sang giá trị số. Cảm biến nhiệt độ được kết nối qua giao tiếp 1-Wire (đối với DS18B20) hoặc ngõ vào analog (đối với LM35). Tín hiệu xử lý được truyền đến module Wifi qua UART, và từ đó gửi dữ liệu lên MQTT Broker.

4.2. Quy trình thực nghiệm

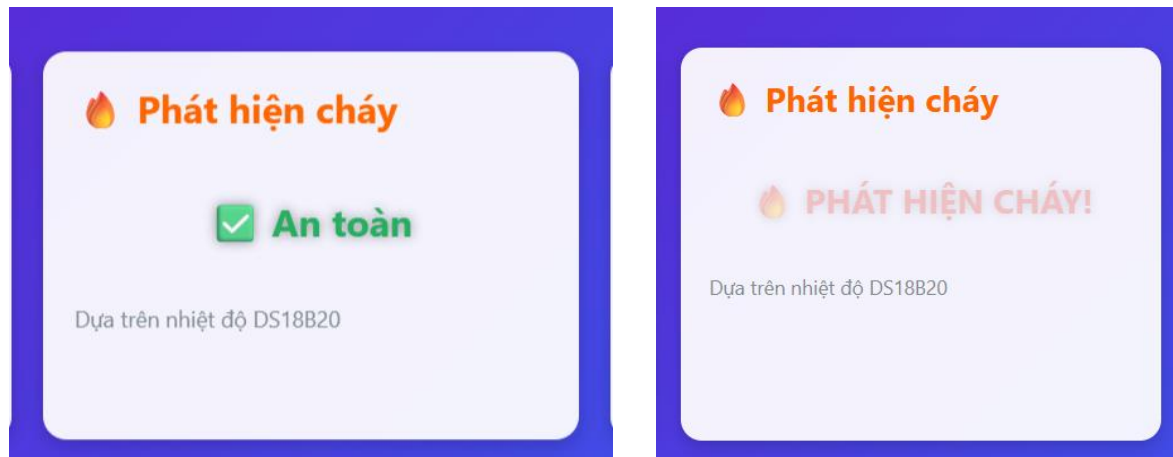
Thực nghiệm được tiến hành trong môi trường phòng thí nghiệm, nguồn cấp ổn định 5 VDC, mạng Wi-Fi nội bộ ít nhiễu. Trước khi thử, hiệu chỉnh ngưỡng IR bằng biến trở trên module ở trạng thái không có lửa, ghi lại giá trị nền; xác định ngưỡng nhiệt độ dựa trên ba kịch bản: môi trường bình thường, đặt nguồn nhiệt cách 30 cm và 60 cm. STM32 chạy chương trình lấy mẫu ở chu kỳ 100 ms và truyền UART 115200 bps. ESP32 kết nối Wi-Fi, thiết lập phiên MQTT và hiển thị trên Dashboard. Thực hiện ba nhóm thử: không có lửa nhưng ánh sáng mạnh chiếu trực tiếp; có ngọn lửa nhỏ (nến) ở các khoảng cách 20, 40, 60, 80 cm; thay đổi nhanh nguồn nhiệt cục bộ để kiểm tra khả năng loại báo giả. Mỗi trường hợp lặp lại tối thiểu mười lần để thu đủ dữ liệu thống kê.



Hình 21: Sơ đồ bố trí thí nghiệm và khoảng cách nguồn lửa

4.3. Kết quả truyền dữ liệu và hiển thị

Trong điều kiện mạng nội bộ, thời gian từ lúc ESP32 nhận khung UART hợp lệ đến khi bản tin MQTT xuất hiện trên Dashboard dao động 120–300 ms. Với khoảng cách 40 cm, giá trị IR tăng vượt ngưỡng trong vòng 100–200 ms sau khi đưa lửa vào vùng nhận; nhiệt độ tăng chậm hơn nhưng đạt ngưỡng xác nhận trong 0.4–0.8 s tùy cường độ nguồn lửa. Khi ánh sáng halogen chiếu trực tiếp, IR tăng nhưng nhiệt độ không đổi nên trạng thái vẫn là an toàn; hệ thống không kích hoạt LED/còi, cho thấy thuật toán xác nhận hai điều kiện hoạt động đúng.



Hình 22: Ảnh chụp Dashboard ở hai trạng thái Safe và Fire Detected –
chèn ảnh

4.4. Đánh giá kết quả

Hệ thống hoạt động ổn định, có khả năng mở rộng và

đáp ứng tốt các tiêu chí kỹ thuật đề ra. Việc kết hợp hai cảm biến cho phép phát hiện cháy chính xác hơn, giảm đáng kể báo động giả. STM32 thực hiện việc thu thập dữ liệu từ cảm biến, trong khi ESP32 xử lý dữ liệu, điều khiển cảnh báo và truyền thông tin qua giao thức MQTT, giúp hệ thống đáp ứng tốt yêu cầu thời gian thực và truyền dữ liệu linh hoạt.

Tuy nhiên, vẫn tồn tại một số hạn chế nhất định. Phạm vi phát hiện của cảm biến IR còn phụ thuộc vào cường độ ngọn lửa và điều kiện ánh sáng môi trường. Cảm biến nhiệt có độ trễ nhất định khi thay đổi nhiệt độ đột ngột, điều này có thể gây chậm phản hồi trong một số trường hợp. Ngoài ra, mô hình hiện tại mới chỉ hoạt động trong mạng Wi-Fi nội bộ, chưa kết nối đến môi trường Internet toàn cầu để giám sát từ xa.

CHƯƠNG 5: KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

5.1. Kết luận

Đề tài “Hệ thống phát hiện ngọn lửa bằng IR kết hợp xác nhận bằng nhiệt độ qua STM32 và MQTT” đã được nghiên cứu, thiết kế và triển khai hoàn chỉnh theo mô hình một hệ thống IoT thu nhỏ. Kết quả cho thấy hệ thống hoạt động ổn định, có khả năng phản ứng nhanh, độ chính xác cao và truyền dữ liệu đáng tin cậy.

Cụ thể, hệ thống được xây dựng gồm khối cảm biến phát hiện ngọn lửa (IR Flame Sensor) và cảm biến nhiệt độ kết nối với vi điều khiển STM32F103C8T6 đảm nhiệm vai trò thu thập và truyền dữ liệu cảm biến sang ESP32 thông qua giao tiếp UART. ESP32 đóng vai trò xử lý trung tâm, thực hiện thuật toán xác nhận hai bước, điều khiển còi – đèn cảnh báo và đồng thời truyền dữ liệu cảnh báo lên MQTT Broker. Mô hình được lắp ráp và thử nghiệm thành công, đảm bảo tính ổn định và độ nhạy cao.

Phần mềm được phát triển trên hai môi trường: STM32Cube IDE (lập trình STM32) và Arduino IDE (lập trình ESP32), sử dụng thư viện HAL và MQTT Client. Hệ thống được kiểm nghiệm với nguồn lửa ở khoảng cách 20–80cm, thời gian phản hồi dưới 1 giây, dữ liệu truyền thành công lên Node-RED Dashboard theo thời gian thực. Kết quả thử nghiệm cho thấy tỷ lệ báo động sai dưới 5%, khẳng định hiệu quả của thuật toán kết hợp cảm biến IR và nhiệt độ.

5.2. Đánh giá chung

Hệ thống đạt được các tiêu chí đề ra về tính ổn định, độ chính xác và khả năng mở rộng. Việc tách riêng vai trò giữa hai vi điều khiển giúp tăng hiệu quả xử lý: STM32 chuyên thu thập dữ liệu cảm biến với độ chính xác cao, trong khi ESP32 chịu trách nhiệm xử lý, điều khiển cảnh báo và kết nối mạng Wifi để truyền dữ liệu qua MQTT. Nhờ vậy, tốc độ phản hồi được cải thiện và hệ thống có khả năng hoạt động liên tục mà không bị trễ hoặc mất kết nối.

Tuy nhiên, hệ thống vẫn có một số hạn chế như phạm vi phát hiện của cảm biến IR phụ thuộc vào cường độ ngọn lửa và điều kiện ánh sáng, cảm biến nhiệt có độ trễ khi thay đổi nhiệt độ nhanh, và việc truyền dữ liệu hiện mới giới hạn trong mạng Wifi nội bộ.

5.3. Hướng phát triển

Trong tương lai, hệ thống có thể được mở rộng theo các hướng sau:

Bổ sung cảm biến khói và khí gas (MQ-2, MQ-5) nhằm nâng cao khả năng phát hiện và phân loại tình huống cháy, từ đó đánh giá mức độ nguy hiểm dựa trên nhiều yếu tố đồng thời.

Kết nối Internet toàn cầu thông qua nền tảng MQTT Cloud (như HiveMQ, Adafruit IO) để người dùng nhận cảnh báo từ xa trên điện thoại di động, tạo thành một hệ thống IoT hoàn chỉnh.

Thiết kế mạch in PCB chuyên dụng để giảm nhiễu, tăng độ bền và thẩm mỹ; có thể thay thế ESP32 bằng bản tích hợp Wi-Fi + Bluetooth hiệu năng cao.

Cải tiến phần mềm Node-RED Dashboard bằng chức năng thống kê dữ liệu, biểu đồ nhiệt độ – thời gian, gửi cảnh báo qua email hoặc Telegram.

Phát triển mô hình mạng lưới cảm biến (Sensor Network), trong đó nhiều STM32 hoạt động như các nút cảm biến gửi dữ liệu về một ESP32 trung tâm để giám sát diện rộng.

5.4. Kết luận chung

Đề tài “Hệ thống phát hiện ngọn lửa bằng IR kết hợp xác nhận bằng nhiệt độ qua STM32 và MQTT” đã hoàn thành toàn bộ mục tiêu nghiên cứu đặt ra. Hệ thống đáp ứng tốt các yêu cầu về độ chính xác, tốc độ phản hồi, độ ổn định truyền thông và khả năng giám sát từ xa. Việc sử dụng STM32 làm bộ thu thập dữ liệu cảm biến và ESP32 làm trung tâm xử lý và truyền MQTT đã chứng minh được hiệu quả và tính linh hoạt cao trong mô hình IoT thực tế.

Kết quả của đề tài không chỉ có giá trị học thuật mà còn mang tiềm năng ứng dụng thực tế cao. Với chi phí thấp, cấu trúc mở, dễ mở rộng và khả năng tương thích tốt với nhiều nền tảng IoT, hệ thống hoàn toàn có thể phát triển thành sản phẩm thương mại phục vụ nhu cầu giám sát an toàn và cảnh báo cháy trong đời sống hiện nay.

TÀI LIỆU THAM KHẢO

- [1] STMicroelectronics, “*STM32F103x8 Datasheet – ARM®-based 32-bit MCU with 64 Kbytes Flash, 72 MHz CPU, ADC, and communication interfaces*,” STMicroelectronics, 2021. [Online]. Available: <https://www.st.com/resource/en/datasheet/stm32f103c8.pdf>
- [2] Maxim Integrated, “*DS18B20 Programmable Resolution 1-Wire Digital Thermometer*,” Dallas Semiconductor, 2019. [Online]. Available: <https://datasheets.maximintegrated.com/en/ds/DS18B20.pdf>
- [3] SparkFun Electronics, “*Flame Sensor Hookup Guide*,” SparkFun, 2020. [Online]. Available: <https://learn.sparkfun.com/tutorials/flame-sensor-hookup-guide>
- [4] P. Thangavel, A. Kumar, and D. Raj, “*IoT based Fire Detection and Notification System using Flame and Temperature Sensors*,” *International Journal of Engineering and Advanced Technology (IJEAT)*, vol. 9, no. 3, pp. 341–345, Feb. 2020.
- [5] A. S. Pasha, “*MQTT Protocol in IoT – An Overview*,” *International Journal of Engineering Research & Technology (IJERT)*, vol. 9, no. 5, pp. 1–4, 2020.
- [6] STMicroelectronics, “*Getting started with STM32CubeIDE*,” Application Note AN5361, 2020. [Online]. Available: https://www.st.com/resource/en/application_note/an5361-getting-started-with-stm32cubeide-stmicroelectronics.pdf
- [7] Nguyễn Văn Hòa, *Giáo trình Vi điều khiển và Ứng dụng*, Nhà xuất bản Đại học Quốc gia TP. Hồ Chí Minh, 2021.
- [8] Trần Đức Trung, *Hệ thống Nhúng và Ứng dụng IoT*, Nhà xuất bản Khoa học và Kỹ thuật, Hà Nội, 2020.
- [9] Lê Ngọc Huy và Nguyễn Thanh Phong, “*Thiết kế mô hình giám sát nhiệt độ và cảnh báo cháy sử dụng giao thức MQTT*,” *Tạp chí Khoa học và Công nghệ – Đại học Đà Nẵng*, số 5(150), tr. 55–62, 2021.

[10] Nguyễn Minh Quân, *Nghiên cứu triển khai MQTT trong hệ thống IoT giám sát môi trường*, Luận văn tốt nghiệp, Trường Đại học Sư phạm Kỹ thuật TP. Hồ Chí Minh, 2022.

[11] L. S. Rodrigues, J. P. Oliveira, and R. R. Souza, “*Design of an IoT Fire Detection System Using STM32 Microcontroller and MQTT Protocol*,” *IEEE Latin America Transactions*, vol. 19, no. 12, pp. 2135–2141, Dec. 2021.

[12] Texas Instruments, “*LM35 Precision Centigrade Temperature Sensors*,” Texas Instruments, 2018. [Online]. Available: <https://www.ti.com/lit/ds/symlink/lm35.pdf>

[13] Bùi Đức Trí, “*Ứng dụng STM32 và MQTT trong giám sát an toàn công nghiệp*,” Báo cáo sinh viên nghiên cứu khoa học, Trường Đại học Sư phạm Kỹ thuật TP. Hồ Chí Minh, 2023.

[14] N. Naik, “*Understanding MQTT for IoT: A Practical Overview*,” *IEEE International Conference on Emerging Trends in Communication Technologies (ICETCT)*, Bangalore, India, 2020.

[15] Node-RED Documentation, “*Getting Started with Node-RED Dashboard*,” IBM, 2021. [Online]. Available: <https://nodered.org/docs/dashboard/>

PHỤC LỤC

CODE STM32

```
#include <WiFi.h>

#include <WiFiClientSecure.h>

#include <PubSubClient.h>

#include <ArduinoJson.h>

#include <OneWire.h>

#include <DallasTemperature.h>


// WiFi Config

const char* WIFI_SSID = "P314";

const char* WIFI_PASS = "P314xinchao";


// MQTT Config

const char* MQTT_SERVER = "c9ca57d1a0c84ad2801f16e7a8e9be63.s1.eu.hivemq.cloud";

const uint16_t MQTT_PORT = 8883;

const char* MQTT_USERNAME = "hivemq.webclient.1761147228563";

const char* MQTT_PASSWORD = ",5Ub2m.L#C:laP7xHT6o";


const char* MQTT_TOPIC = "home/fire_sensor/1";

const char* MQTT_CONTROL_TOPIC = "home/fire_control/1";


// Pin definitions

#define FLAME_PIN 25

#define ONE_WIRE_BUS 26

#define LED_PIN 2

#define BUZZER_PIN 19


// Globals
```

```

WiFiClientSecure espClient;
PubSubClient client(espClient);

OneWire oneWire(ONE_WIRE_BUS);
DallasTemperature sensors(&oneWire);

bool flameDetected = false;
float temperature = 0.0;
bool alarmActive = false;
unsigned long lastPublish = 0;

// Custom log timestamp
String getTimestamp() {
    unsigned long ms = millis();
    unsigned long seconds = ms / 1000;
    unsigned long minutes = seconds / 60;
    unsigned long hours = minutes / 60;
    ms = ms % 1000;
    seconds = seconds % 60;
    minutes = minutes % 60;
    hours = hours % 24;

    char buffer[30];
    sprintf(buffer, "%02lu:%02lu:%02lu,%03lu", hours, minutes, seconds, ms);
    return String(buffer);
}

// ===== MQTT CALLBACK =====

```

```

void mqttCallback(char* topic, byte* payload, unsigned int length) {
    Serial.printf("%s [MQTT] Client esp32 received PUBLISH (Mid: 1, RC:0)\n",
getTimestamp().c_str());

    String message;
    for (int i = 0; i < length; i++) {
        message += (char)payload[i];
    }

    StaticJsonDocument<200> doc;
    if (!deserializeJson(doc, message)) {
        if (doc.containsKey("command") && doc["command"] == "reset") {
            digitalWrite(LED_PIN, LOW);
            digitalWrite(BUZZER_PIN, LOW);
            alarmActive = false;
        }
        if (doc.containsKey("buzzer")) {
            String state = doc["buzzer"];
            digitalWrite(BUZZER_PIN, state == "on" ? HIGH : LOW);
        }
    }
}

// ===== SETUP =====

void setup() {
    Serial.begin(115200);
    delay(1000);
}

```

```

pinMode(FLAME_PIN, INPUT);
pinMode(LED_PIN, OUTPUT);
pinMode(BUZZER_PIN, OUTPUT);
digitalWrite(LED_PIN, LOW);
digitalWrite(BUZZER_PIN, LOW);

sensors.begin();

// WiFi
WiFi.begin(WIFI_SSID, WIFI_PASS);
while (WiFi.status() != WL_CONNECTED) {
    delay(500);
}

// SSL
espClient.setInsecure();

// MQTT
client.setServer(MQTT_SERVER, MQTT_PORT);
client.setCallback(mqttCallback);
client.setKeepAlive(60);
}

// ===== LOOP =====

void loop() {
    if (!client.connected()) {
        reconnectMQTT();
    }
}

```



```

client.loop();

readSensors();
controlAlarm();

if (millis() - lastPublish > 5000) {
    publishMQTT();
    lastPublish = millis();
}
}

// ===== RECONNECT =====

void reconnectMQTT() {
    while (!client.connected()) {

        String clientId = "firebase_subscriber";

        // Log CONNECT
        Serial.printf("%s [MQTT] Sending CONNECT (u1, p1, wr0, wq0, wf0, c0, k60)
client_id=b'%s'\n",
                        getTimestamp().c_str(), clientId.c_str());

        Serial.printf("🔗 Listening for logs from ESP32... (persistent session ON, QoS=1)\n\n");

        if (client.connect(clientId.c_str(), MQTT_USERNAME, MQTT_PASSWORD, NULL, 0,
true, NULL, true)) {

            // Log CONNACK
            Serial.printf("%s [MQTT] Received CONNACK (1, 0)\n", getTimestamp().c_str());

            Serial.printf("✅ Connected to MQTT broker successfully.\n");

```

```

// Log SUBSCRIBE
Serial.printf("%s [MQTT] Sending SUBSCRIBE (d0, m1) [(b'%s', 1)]\n",
              getTimestamp().c_str(), MQTT_CONTROL_TOPIC);
client.subscribe(MQTT_CONTROL_TOPIC, 1);

// Log SUBACK
Serial.printf("%s [MQTT] Received SUBACK\n", getTimestamp().c_str());
Serial.printf("🔔 Subscribed to topics: %s\n\n", MQTT_CONTROL_TOPIC);

} else {
    Serial.printf("%s [MQTT] Connection failed, rc=%d\n", getTimestamp().c_str(),
client.state());
    delay(5000);
}
}
}

// ===== PUBLISH =====

void publishMQTT() {
    StaticJsonDocument<300> doc;
    doc["flame"] = flameDetected;
    doc["temperature"] = temperature;
    doc["alarm"] = alarmActive;
    doc["timestamp"] = millis();
    doc["device"] = "ESP32_FireAlarm";

    char buffer[300];
    size_t n = serializeJson(doc, buffer);

```

```

// Log PUBLISH
Serial.printf("%s [MQTT] Client esp32 sending PUBLISH (d0, q1, r0, m1, '%s', ... (%d
bytes))\n",
            getTimestamp().c_str(), MQTT_TOPIC, n);

if (client.publish(MQTT_TOPIC, buffer, false)) {
    // Log PUBACK (nếu QoS=1)
    Serial.printf("%s [MQTT] Client esp32 received PUBACK (Mid: 1, RC:0)\n",
getTimestamp().c_str());
}
}

// ===== READ SENSORS =====

void readSensors() {
    flameDetected = (digitalRead(FLAME_PIN) == LOW);
    sensors.requestTemperatures();
    float tempRead = sensors.getTempCByIndex(0);
    if (tempRead != -127.0 && tempRead != 85.0) {
        temperature = tempRead;
    }
}

// ===== CONTROL ALARM =====

void controlAlarm() {
    if (flameDetected || temperature > 50.0) {
        alarmActive = true;
        static unsigned long lastBlink = 0;

```

```

if (millis() - lastBlink > 300) {
    lastBlink = millis();
    digitalWrite(LED_PIN, !digitalRead(LED_PIN));
    digitalWrite(BUZZER_PIN, !digitalRead(BUZZER_PIN));
}
} else {
    alarmActive = false;
    digitalWrite(LED_PIN, LOW);
    digitalWrite(BUZZER_PIN, LOW);
}
}

```

CODE WEB

```

<!DOCTYPE html>
<html lang="vi">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>🔥 ESP32 Fire Detection Dashboard</title>
<script src="https://unpkg.com/mqtt/dist/mqtt.min.js"></script>
<style>
body {
    font-family: "Segoe UI", sans-serif;
    background: linear-gradient(135deg, #6a11cb, #2575fc);
    color: #fff;
    margin: 0;
    padding: 0;
}

h1 {

```

```
text-align: center;
padding: 20px;
font-size: 28px;
color: #fff;
text-shadow: 0 0 8px #0006;
}

.container {
display: flex;
flex-wrap: wrap;
justify-content: center;
gap: 20px;
padding: 20px;
}

.card {
background: #ffffffee;
color: #222;
border-radius: 15px;
padding: 20px;
width: 300px;
box-shadow: 0 4px 12px #0003;
transition: transform 0.3s;
}

.card:hover {
transform: translateY(-5px);
}
```

```
.card h2 {  
  margin-top: 0;  
  color: #ff6600;  
}  
  
.temp {  
  font-size: 48px;  
  font-weight: bold;  
  color: #27ae60;  
  transition: color 0.3s;  
}  
  
.temp.hot {  
  color: #e74c3c;  
  animation: pulse 1s infinite;  
}  
  
@keyframes pulse {  
  0%, 100% { opacity: 1; }  
  50% { opacity: 0.7; }  
}  
  
.safe {  
  color: #27ae60;  
  font-weight: bold;  
  font-size: 24px;  
}  
  
.danger {
```

```
color: #e74c3c;
font-weight: bold;
font-size: 24px;
animation: blink 1s infinite;
}
```

```
@keyframes blink {
  0%, 50% { opacity: 1; }
  51%, 100% { opacity: 0.3; }
}
```

```
button {
  margin: 5px;
  padding: 12px 20px;
  border: none;
  border-radius: 10px;
  cursor: pointer;
  font-weight: bold;
  transition: 0.2s;
  font-size: 14px;
}
```

```
button:hover {
  opacity: 0.85;
  transform: scale(1.05);
}
```

```
button:active {
  transform: scale(0.98);
}
```

```

}

.on { background: #e74c3c; color: #fff; }
.off { background: #2ecc71; color: #fff; }
.reset { background: #3498db; color: #fff; }

.status-indicator {
  display: inline-block;
  width: 12px;
  height: 12px;
  border-radius: 50%;
  background: #95a5a6;
  margin-left: 8px;
  animation: blink-slow 2s infinite;
}

.status-indicator.connected {
  background: #2ecc71;
  box-shadow: 0 0 10px #2ecc71;
}

@keyframes blink-slow {
  0%, 100% { opacity: 1; }
  50% { opacity: 0.5; }
}

/* --- Nhật ký --- */
.log-box {
  width: 640px;

```



```
max-height: 200px;
overflow-y: auto;
background: #ffffffcc;
color: #111;
font-family: monospace;
border-radius: 10px;
padding: 10px;
box-shadow: inset 0 0 10px #0003;
line-height: 1.6;
font-size: 13px;
}

.timestamp {
  color: #555;
  font-size: 11px;
  font-weight: bold;
}

.log-info { color: #007acc; }
.log-success { color: #16a34a; }
.log-warn { color: #eab308; }
.log-error { color: #dc2626; font-weight: bold; }
.log-default { color: #333; }

.footer {
  text-align: center;
  padding: 10px;
  color: #eee;
  font-size: 14px;
```

```

}

.threshold-badge {
  display: inline-block;
  background: #f39c12;
  color: white;
  padding: 5px 12px;
  border-radius: 20px;
  font-size: 13px;
  font-weight: bold;
  margin-top: 10px;
}

@media (max-width: 700px) {
  .card, .log-box {
    width: 90%;
  }
}
</style>
</head>
<body>

<h1><img alt="ESP32 logo" data-bbox="188 715 208 735"/> HỆ THỐNG GIÁM SÁT CHÁY ESP32 <span class="status-indicator"
id="statusDot"></span></h1>

<div class="container">
  <div class="card">
    <h2><img alt="Thermometer icon" data-bbox="208 855 228 875"/> Nhiệt độ hiện tại</h2>
    <div class="temp" id="temp">--°C</div>
  </div>
</div>

```

```

<div class="threshold-badge">Ngưỡng: <span id="threshold">50</span>°C</div>

<p style="margin-top: 10px; color: #7f8c8d; font-size: 13px;">⚠ Cảnh báo khi vượt
ngưỡng</p>

</div>

<div class="card">

<h2>🔥 Phát hiện cháy</h2>

<h1 id="flameStatus" class="safe">✅ An toàn</h1>

<p style="color: #7f8c8d; font-size: 14px;">Dựa trên nhiệt độ DS18B20</p>

</div>

<div class="card">

<h2>🔔 Điều khiển</h2>

<p>Trạng thái: <span id="alarmStatus" class="safe">Bình thường</span></p>

<button id="resetBtn" class="reset">🔄 Reset Alarm</button><br>

<button id="buzzerOn" class="on">🔊 Buzzer ON</button>

<button id="buzzerOff" class="off">🔇 Buzzer OFF</button>

</div>

<div class="card" style="width:640px">

<h2>📝 Nhật ký hoạt động</h2>

<div id="log" class="log-box"></div>

<button onclick="clearLog()" style="margin-top: 10px; background: #95a5a6; color: white;
padding: 8px 16px; border-radius: 6px; border: none; cursor: pointer;">🗑 Xóa log</button>

</div>

</div>

<div class="footer">

```

 Kết nối MQTT với HiveMQ Cloud | Sensor: DS18B20 | Made with ❤️ by Nam thuộc nhóm 7

</div>

<script>

// MQTT Config

const broker = "wss://c9ca57d1a0c84ad2801f16e7a8e9be63.s1.eu.hivemq.cloud:8884/mqtt";

const username = "hivemq.webclient.1761147228563";

const password = ",5Ub2m.L#C:laP7xHT6o";

const subTopic = "home/fire_sensor/1";

const pubTopic = "home/fire_control/1";

const client = mqtt.connect(broker, {

username,

password,

reconnectPeriod: 2000,

clean: true,

connectTimeout: 4000

});

const logBox = document.getElementById("log");

let lastTemp = null;

function addLog(msg, type = "default") {

const time = new Date().toLocaleTimeString();

const p = document.createElement("div");

p.classList.add("log-" + type);

p.innerHTML = `<span class="timestamp">[\${time}]</span> \${msg}`;

logBox.prepend(p);

```

// Giới hạn 100 log
if (logBox.children.length > 100) {
    logBox.removeChild(logBox.lastChild);
}
}

function clearLog() {
    logBox.innerHTML = "";
    addLog("🗑️ Đã xóa log", "info");
}

client.on("connect", () => {
    addLog("✅ Kết nối MQTT thành công!", "success");
    document.getElementById("statusDot").classList.add("connected");
    client.subscribe(subTopic);
});

client.on("reconnect", () => {
    addLog("🔄 Đang thử kết nối lại MQTT...", "warn");
    document.getElementById("statusDot").classList.remove("connected");
});

client.on("error", (err) => {
    addLog("❌ Lỗi MQTT: " + err.message, "error");
    document.getElementById("statusDot").classList.remove("connected");
});

client.on("message", (topic, message) => {

```

```

try {
    const data = JSON.parse(message.toString());

    // Cập nhật nhiệt độ
    const temp = parseFloat(data.temperature);
    const tempEl = document.getElementById("temp");
    tempEl.textContent = temp.toFixed(1) + "°C";

    // Đổi màu nhiệt độ
    const threshold = data.threshold || 50;
    document.getElementById("threshold").textContent = threshold;

    if (temp > threshold) {
        tempEl.classList.add("hot");
    } else {
        tempEl.classList.remove("hot");
    }

    // Cập nhật trạng thái lửa (dựa vào flame field)
    const flameStatusEl = document.getElementById("flameStatus");
    if (data.flame) {
        flameStatusEl.textContent = "🔥 PHÁT HIỆN CHÁY!";
        flameStatusEl.className = "danger";
    } else {
        flameStatusEl.textContent = "✅ An toàn";
        flameStatusEl.className = "safe";
    }

    // Cập nhật trạng thái alarm

```

```

const alarmStatusEl = document.getElementById("alarmStatus");

if (data.alarm) {

    alarmStatusEl.textContent = "🚨 Đang báo động!";

    alarmStatusEl.className = "danger";

} else {

    alarmStatusEl.textContent = "Bình thường";

    alarmStatusEl.className = "safe";

}


// Log các sự kiện quan trọng

if (data.flame && (!lastTemp || !lastTemp.flame)) {

    addLog(`🔥🔥🔥 CẢNH BÁO: Phát hiện cháy! Nhiệt độ: ${temp.toFixed(1)}°C`,
"error");

    playAlertSound();

} else if (!data.flame && lastTemp && lastTemp.flame) {

    addLog(`✅ Nhiệt độ trở lại bình thường: ${temp.toFixed(1)}°C`, "success");

} else if (Math.abs(temp - (lastTemp?.temperature || temp)) > 5) {

    addLog(`📊 Nhiệt độ thay đổi đáng kể: ${temp.toFixed(1)}°C`, "warn");

} else {

    addLog(`📧 Dữ liệu: 🌡️=${temp.toFixed(1)}°C | 🔥=${data.flame} | 🚨=${data.alarm}`, "info");

}


lastTemp = data;


} catch (e) {

    addLog("❌ Lỗi parse dữ liệu: " + e.message, "error");

}

});

```

```

function sendCommand(cmd) {
  const msg = JSON.stringify(cmd);
  client.publish(pubTopic, msg);
  addLog("🔊 Gửi lệnh: " + JSON.stringify(cmd), "warn");
}

function playAlertSound() {
  try {
    const audioContext = new (window.AudioContext || window.webkitAudioContext)();
    const oscillator = audioContext.createOscillator();
    const gainNode = audioContext.createGain();

    oscillator.connect(gainNode);
    gainNode.connect(audioContext.destination);

    oscillator.frequency.value = 800;
    oscillator.type = 'square';

    gainNode.gain.setValueAtTime(0.3, audioContext.currentTime);
    gainNode.gain.exponentialRampToValueAtTime(0.01, audioContext.currentTime + 0.5);

    oscillator.start(audioContext.currentTime);
    oscillator.stop(audioContext.currentTime + 0.5);
  } catch (e) {
    console.log("Cannot play sound:", e);
  }
}

```



```
document.getElementById("resetBtn").onclick = () => sendCommand({ command: "reset"
});
document.getElementById("buzzerOn").onclick = () => sendCommand({ buzzer: "on" });
document.getElementById("buzzerOff").onclick = () => sendCommand({ buzzer: "off" });
</script>

</body>
</html>
```